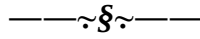


VIETNAM NATIONAL UNIVERSITY

INTERNATIONAL SCHOOL



FINAL REPORT

Blockchain and Digital Currency

Topic 1: Decentralized Voting System Based on Blockchain Technology

Group: 9

Lecturer MSc. Ha Thi Kim Dung

Students Nguyen Quoc Huy - 23070110

Nguyen Nam Khanh - 23070291

Phan Tuan Minh - 23071114

Hanoi, 2026

MEMBER

NAME	STUDENT CODE	CONTRIBUTE
Nguyen Quoc Huy	23070110	100%
Nguyen Nam Khanh	23070291	100%
Phan Tuan Minh	23071114	100%

Contents

1. Introduction	8
1.1. Background	8
1.2. Problem Statement	8
1.3. Objectives.....	9
1.4. Scope of the Project.....	10
1.4.1. Technical Scope	10
1.4.2. Functional Scope.....	10
1.5. Report Organization	11
2. Background and Related Technologies	11
2.1. Overview of Blockchain.....	11
2.2. Smart Contract Development Technology	12
2.3. Web Application Development Technology	13
3. System Analysis.....	14
3.1 System Overview	14
3.2 Actors	15
3.3 User Story:.....	16
3.4 Business Process Model and Notation	19
3.5 Functional Requirements.....	20
3.6 Non-Functional Requirements	27
3.7 Use Case Diagram.....	33
3.8 Main Workflow	38
3.9 System Design.....	41
3.9.1 Overall Architecture.....	41
3.9.2 Database Design	42
3.9.3 Data Design.....	43
3.9.4 Smart Contract Design.....	45
3.9.5 User Interface Design	49

3.9.6 Security and Privacy Design	53
4. Implementation and Software Demo	56
4.1 Development Environment	56
4.2. Smart Contract Implementation	59
4.3 Main Screenshots	60
4.4 Demo Scenario	66
4.5 Testing.....	69
5. Conclusion.....	73
5.1. Advantages of the Proposed System	73
5.2. Limitations	74
5.3. Future Work	75
6. References	77

List of Tables

Table 1: List of Abbreviations	7
Table 2: Actor Permission Matrix	15
Table 3: User Stories	19
Table 4: Functional Requirements.....	27
Table 5: Non-Functional Requirements - Performance	27
Table 6: Non-Functional Requirements - Security.....	29
Table 7: Non-Functional Requirements - Reliability and Availability	29
Table 8: Non-Functional Requirements - Usability	30
Table 9: Non-Functional Requirements - Compatibility	31
Table 10: Non-Functional Requirements - Maintainability	31
Table 11: Non-Functional Requirements - Transparency and Auditability	32
Table 12: Non-Functional Requirements - Scalability	33

Table 13: Main Data Description	45
Table 14: Candidate Data Structure.....	45
Table 15: Election Data Structure	46
Table 16: Main State Variables	47
Table 17: Smart Contract Modifiers	47
Table 18: Smart Contract Member Functions.....	48
Table 20: Security and Privacy Design	56
Table 21: Development Environment	59
Table 22: Smart Contract Functions	60
Table 23: Demo Scenario	69
Table 24: Testing Results	73

List of Figures

Figure 1: System Context Diagram	14
Figure 2: Business Process Model and Notation Diagram.....	20
Figure 3: Use Case Diagram of the Decentralized Voting System	34
Figure 4: Submit Data Process Use Case Diagram	34
Figure 5: Verify Data Process Use Case Diagram	35
Figure 6: Create Transaction Process Use Case Diagram.....	36
Figure 7: View History Process Use Case Diagram.....	37
Figure 8: Use Case Distribution by Actor and Execution Layer	38
Figure 9: Main Workflow of the Decentralized Voting System.....	41
Figure 10: Overall System Architecture.....	42
Figure 11: Entity Relationship Diagram	43
Figure 12: Login/Register Interface Design	50
Figure 13: Admin Dashboard Interface Design	50

Figure 14: Create Election Modal Interface Design	51
Figure 15: Add Candidates and Election Detail Interface Design	51
Figure 16: Voter Voting Interface Design	51
Figure 17: Vote History Interface Design	52
Figure 18: Election Results Interface Design	52
Figure 19: Blockchain Explorer Interface Design.....	53
Figure 20: Login/Register Screen.....	60
Figure 21: Admin Dashboard Screen.....	61
Figure 22: Election Management Screen	61
Figure 23: User Management Screen.....	62
Figure 24: Create Election Modal	63
Figure 25: Add Candidates / Election Detail Screen	63
Figure 26: Voting Screen.....	64
Figure 27: Vote History Screen.....	64
Figure 28: Election Results Screen.....	65
Figure 29: Blockchain Explorer Screen	66

List of Abbreviations

Abbreviation	Full Form
ABI	Application Binary Interface
API	Application Programming Interface
BPMN	Business Process Model and Notation
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DApp	Decentralized Application
DB	Database
DVS	Decentralized Voting System
ERD	Entity Relationship Diagram
ETH	Ether
EVM	Ethereum Virtual Machine
FR	Functional Requirement
JWT	JSON Web Token
JSON	JavaScript Object Notation
LTS	Long-Term Support
NFR	Non-Functional Requirement
NFT	Non-Fungible Token
OS	Operating System
RBAC	Role-Based Access Control
SPA	Single Page Application
SQL	Structured Query Language
SQLite	SQLite Database Engine
Tx / tx	Transaction
tx_hash	Transaction Hash
UI	User Interface
UX	User Experience
Web2	Traditional Web Application Layer
Web3	Decentralized Web Layer

Table 1: List of Abbreviations

1. Introduction

1.1. Background

In today's digital era, the digitization of traditional systems is accelerating rapidly across the globe. However, the field of voting and elections still faces critical challenges that have not been thoroughly resolved by conventional digital systems.

Blockchain technology, introduced alongside Bitcoin in 2009 by Satoshi Nakamoto, opened a new era in decentralized data storage and verification. Blockchain is a distributed ledger that records transactions chronologically, links them using cryptographic algorithms, and replicates them across thousands of network nodes worldwide. The most prominent characteristic of Blockchain is its immutability, meaning that once data is written to the chain, it cannot be deleted or modified by any party, including the system creators themselves.

Ethereum, launched in 2015 by Vitalik Buterin, elevated the concept of Blockchain by introducing Smart Contracts, which are self-executing computer programs stored and run directly on the Blockchain. Smart Contracts allow complex business rules to be encoded and automatically executed without a trusted intermediary. This creates an ideal foundation for building Decentralized Applications, commonly known as DApps, across various fields, including elections and voting.

VoteChain was born out of this context as a decentralized voting system that fully integrates Ethereum Blockchain technology using the Sepolia Testnet, a Node.js and Express.js backend, an SQLite database, and a React.js user interface with MetaMask wallet integration. The system aims to provide a modern, transparent, secure, and accessible voting platform for all types of users.

1.2. Problem Statement

Current voting systems, whether traditional paper-based methods or standard digital systems, suffer from serious unresolved issues.

In traditional voting systems, the vote-counting process relies entirely on the integrity of the organizing committee, preventing voters from independently verifying that their ballots were accurately recorded and unaltered. Similarly, conventional digital systems utilize centralized databases where data can be modified by system administrators without leaving a trace, forcing users to blindly trust the organizing entity without an independent verification mechanism.

Centralized electronic voting systems are frequent targets for hackers, and a successful attack on a central server can alter election results without leaving definitive evidence. Furthermore, the issue of double voting, if not strictly controlled at the technical level, can easily be exploited to manipulate outcomes.

An ideal voting system must simultaneously verify that each voter casts exactly one vote and keep the voter's choice completely secret to ensure anonymity. Current systems usually only meet one of these two conflicting requirements.

Organizing a traditional election incurs significant costs for printing, transportation, ballot-counting personnel, and document storage. For small and medium-sized organizations such as businesses, schools, or communities, this financial burden represents a major barrier that causes important decisions to be made subjectively rather than through transparent voting.

Once an election concludes, conducting an independent audit of the process is highly challenging. Physical records can be lost, damaged, or altered, while conventional digital systems often clear system logs after results are announced, eliminating future auditing capabilities.

1.3. Objectives

The primary objective of the VoteChain project is to design and implement a complete, fully functional decentralized voting platform that covers the entire lifecycle of an election, from initial setup to the final publication of results. To achieve a secure and reliable environment, the project focuses on four key operational goals.

First, the system aims to ensure absolute transparency and independent auditability by recording all critical election actions permanently on a public ledger, allowing any third party to verify the data. Second, it resolves the fundamental challenge of ballot anonymity, utilizing cryptographic mechanisms to verify voter participation and completely prevent double voting without exposing individual candidate selections. Third, the project implements a robust multi-layer authentication framework that secures the platform at both the application and network levels. Finally, the system delivers a comprehensive administrative dashboard alongside internal data tracking tools to support efficient operation, high scannability, and seamless user interaction throughout the voting process.

1.4. Scope of the Project

1.4.1. Technical Scope

- **Smart Contract Layer:** The system deploys a single Smart Contract (Voting.sol) written in Solidity version ^0.8.24, deployed on the Ethereum Sepolia Testnet. The contract manages all on-chain voting logic, including creating elections, managing candidates, receiving votes, and locking results. The system is not deployed on the Ethereum Mainnet due to real gas fee constraints.
- **Backend Layer:** The server is built with Node.js and Express.js, handling the storage of metadata (election names, descriptions, candidate details) and transaction receipts in a local SQLite database. The backend does not execute any voting logic, all voting core logic is delegated to the Smart Contract. API security is enforced using JWT (JSON Web Token) and bcryptjs for password hashing.
- **Frontend Layer:** The user interface is built using React.js (Vite build tool) with a modern Dark Mode design. It integrates Ethers.js v6 to communicate with MetaMask and the Smart Contract. System routing utilizes React Router v6 with a Protected Routes mechanism to differentiate between Admin and Voter roles.

1.4.2. Functional Scope

In-scope:

- Account registration and login with Admin/Voter role separation
- MetaMask wallet connection and synchronization with the system account
- Election creation and management (Admin)
- Candidate addition to elections (Admin)
- Anonymous voting on the Blockchain (Voter)
- Real-time election result viewing with interactive charts
- Transaction lookups and verification using Transaction Hashes
- Personal voting history review
- Finalization and publication of final results (Admin)

Out-of-scope:

- Real payment systems using ETH with real-world value (only using test Sepolia ETH)
- Production environment deployment with a real domain name
- Native Mobile Applications (iOS/Android)
- Automated Email notification features
- Post-voting NFT Badge integration (proposed for the next version)
- Multi-language support (only Vietnamese is supported)

1.5. Report Organization

The report is structured into five distinct chapters following standard software engineering documentation guidelines. Chapter 1 presents the introduction, detailing the project background, problem statement, objectives, scope, and overall report structure. Chapter 2 covers the literature review and related technologies, providing foundational knowledge of Blockchain and Ethereum alongside the rationale for the chosen technology stack. Chapter 3 explains the system design and architecture, presenting the 3-tier architecture, database schema, Smart Contract design, Use Case Diagrams, Sequence Diagrams, and Data Flow Diagrams. Chapter 4 details the implementation and testing phase, describing the development process, technical challenges, and testing results based on User Stories. Finally, Chapter 5 provides the conclusion and future work, summarizing achievements and proposing future development paths.

2. Background and Related Technologies

2.1. Overview of Blockchain

- Definition and Core Concepts:

Blockchain is a decentralized, immutable, and transparent distributed ledger technology that records data in cryptographically linked blocks across a peer-to-peer network. First introduced with Bitcoin by Satoshi Nakamoto, blockchain removes the need for a central authority by replicating data across many independent nodes. Its key characteristics include decentralization, immutability, transparency, and resistance to data manipulation.

- Ethereum and Smart Contracts:

Ethereum expanded blockchain beyond digital currency by introducing smart contracts, which are self-executing programs stored and executed on the blockchain. Smart contracts allow business rules to run automatically without trusted intermediaries. Ethereum uses Ether and Gas to manage computational resources and maintain network operations.

- *Sepolia Testnet:*

Sepolia is an Ethereum test network that simulates Mainnet behavior without using real financial value. It allows developers to deploy contracts, test transactions, verify wallet interactions, and observe block confirmations safely. VoteChain operates on Sepolia to evaluate blockchain voting functions with no real financial risk while maintaining public verifiability.

- *Blockchain in Electronic Voting:*

Blockchain is highly suitable for electronic voting because it supports tamper-resistant records, public auditability, and transparent result verification. However, blockchain voting still faces challenges such as user experience, gas optimization, and voter privacy. VoteChain addresses these issues by combining cryptographic hashing for anonymous ballot protection with a wallet-based interface for easier user interaction.

2.2. Smart Contract Development Technology

- *Solidity Programming Language:*

Solidity is used to implement the core logic of VoteChain's smart contract. It supports secure election and candidate structures, voter tracking, double-voting prevention, access control, and event logging. These features allow the system to enforce voting rules directly on-chain while enabling external components to read blockchain data efficiently.

- *Hardhat Development Environment:*

Hardhat is used to compile, test, debug, and deploy the smart contract. It provides a local blockchain environment for development and supports automated deployment to the Sepolia testnet. This helps reduce configuration errors and improves the reliability of contract testing and deployment.

- *Vote Anonymity Through Cryptographic Hashing:*

VoteChain uses cryptographic hashing to protect voter privacy while preserving verifiability. When a vote is submitted, the system generates a unique vote receipt from election

data, candidate data, voter wallet address, and timestamp. The resulting hash proves voting participation but cannot be reversed to reveal the voter's selected candidate.

2.3. Web Application Development Technology

- *React.js and Component-Based UI Architecture:*

The VoteChain interface follows a component-based design to organize screens and functions clearly. It manages user authentication, wallet status, and blockchain interaction states, while protected routing ensures that restricted functions are only accessible to authorized users.

- *Ethers.js Library:*

Ethers.js connects the user interface with the blockchain network. It helps read blockchain data, estimate transaction costs, track network states, and send transaction requests to the user's wallet for signing. It also handles blockchain-specific data formats safely.

- *Node.js and Express.js Backend Framework:*

The backend manages API requests, authentication, business logic, and communication with the database. Its route, middleware, and controller structure separates responsibilities clearly, making the system easier to maintain and extend.

- *SQLite Database and Data Persistence Strategy:*

SQLite stores off-chain data such as user accounts, roles, election metadata, candidate profiles, and transaction records. This improves data retrieval speed and reduces unnecessary blockchain queries. Transaction hash records are also stored to support auditing and verification.

- *JSON Web Token Authentication:*

User sessions are managed through token-based authentication. After successful login, the system issues a signed token containing user identity and role information. This token is used to verify access rights on protected requests without repeatedly querying the database.

- *MetaMask Wallet Integration:*

MetaMask allows users to manage private keys securely and sign blockchain transactions without exposing sensitive wallet information to the application. It is used for wallet

connection, transaction confirmation, account monitoring, and network switching to ensure users interact with the correct blockchain network.

3. System Analysis

3.1 System Overview

System Context Diagram of the VoteChain system describes the relationship between the central system and external actors. At the center is the VoteChain System, which includes the user interface, backend API, and local database cache. The system exchanges data with the System Admin by receiving login information, election setup data, candidate lists, and management commands; in return, it provides authentication status, operation feedback, and election statistics reports.

For Voter/Citizen, the system receives registration and login information, election viewing requests, voting data, and receipt verification requests. It then returns election lists, candidate information, vote confirmation, digital receipts, and public results. In addition, VoteChain interacts with Web3 Wallet/MetaMask to connect wallets, retrieve public wallet addresses, and sign transactions. Important actions such as creating elections, registering voters, casting votes, and querying data are sent to the Blockchain Network to ensure transparency, immutability, and verifiability. Overall, VoteChain acts as a coordinating system between users, digital wallets, blockchain, and administrative data.

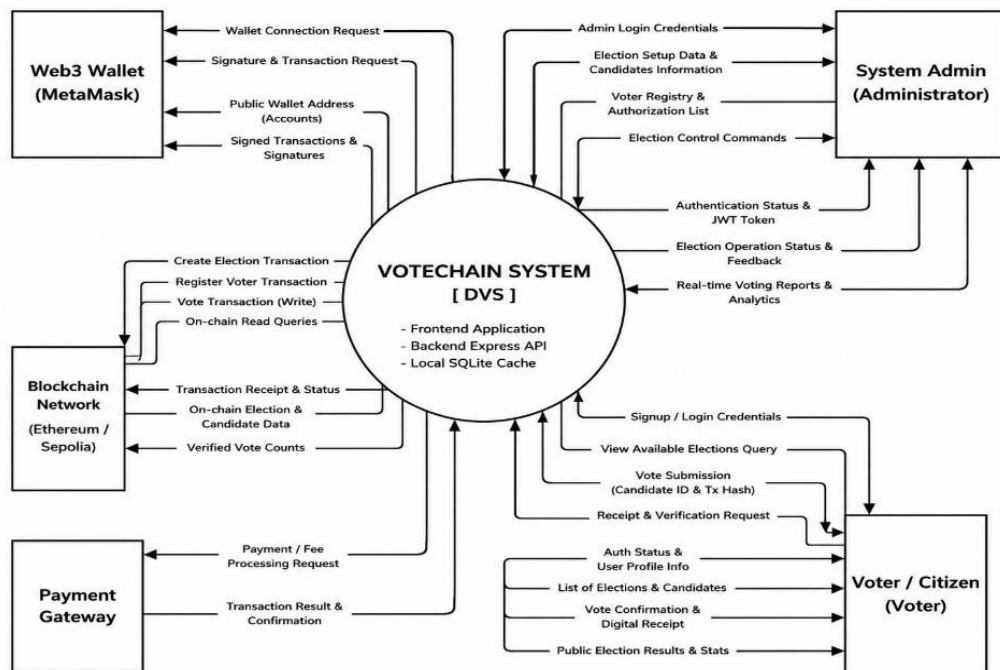


Figure 1: System Context Diagram

3.2 Actors

Feature	Admin	Voter	Public
Register / Login	Yes	Yes	No
Connect MetaMask	Yes	Yes	No
Create Election	Yes	No	No
Add Candidate	Yes	No	No
Finalize Election	Yes	No	No
Delete Election	Yes (Pending only)	No	No
View Dashboard Stats	Yes	No	No
View System Activity Log	Yes	No	No
Cast Vote	Yes	Yes	No
View Election Results	Yes	Yes	Yes
View Personal Vote History	Yes	Yes	No
Search Transaction Hash	Yes	Yes	Yes
Verify on Etherscan	Yes	Yes	Yes

Table 2: Actor Permission Matrix

Admin

- Authentication: Email/Password + JWT Token + MetaMask Private Key (2-layer)
- Responsibilities: Deploy smart contract, create elections, add candidates, finalize results, monitor system activity via dashboard
- Access Scope: All admin pages, all API endpoints, Smart Contract owner functions

Voter

- Authentication: Email/Password + JWT Token + MetaMask wallet connection

- Responsibilities: Browse active elections, select candidate, cast vote on-chain, view personal vote history, verify transaction receipts
- Access Scope: Voter pages only, read APIs, castVote() contract function

Public Visitor

- Authentication: None required
- Responsibilities: View election results, verify any transaction hash independently via Etherscan
- Access Scope: Public result pages only, no write operations

3.3 User Story:

ID	Epic	Title	User Story
US-01	Authentication & Wallet	Register Account	As a new user, I want to register an account with my username, email and password, so that I can access the VoteChain voting platform.
US-02	Authentication & Wallet	Login to System	As a registered user, I want to log in using my email and password, so that I can access my personalized dashboard and voting features.
US-03	Authentication & Wallet	Stay Logged In After Refresh	As a logged-in user, I want my session to persist after refreshing the page, so that I don't have to log in every time I revisit the app.
US-04	Authentication & Wallet	Connect MetaMask Wallet	As a user, I want to connect my MetaMask wallet to my account, so that I can interact with the Blockchain and participate in on-chain voting.

US-05	Authentication & Wallet	Auto-sync Wallet on Account Switch	As a user, I want my wallet address to automatically update when I switch accounts in MetaMask, so that my profile always reflects the correct wallet.
US-06	Admin - Election Management	Create a New Election	As an Admin, I want to create a new election with a name, description, start time, and end time on the Blockchain, so that voters can participate in a transparent and immutable voting event.
US-07	Admin - Election Management	View All Elections	As an Admin, I want to view a list of all elections with their real-time statuses, so that I can monitor and manage all ongoing and past elections.
US-08	Admin - Election Management	Filter Elections by Status	As an Admin, I want to filter elections by their status, so that I can quickly find elections that need my attention.
US-09	Admin - Election Management	Add Candidates to an Election	As an Admin, I want to add candidates to an election before voting starts, so that voters have a list of options to choose from.
US-10	Admin - Election Management	Finalize an Election	As an Admin, I want to finalize a completed election on the Blockchain, so that results are permanently locked and publicly verifiable.
US-11	Admin - Election Management	Delete a Pending Election	As an Admin, I want to delete an election that hasn't started yet, so that I can clean up test or incorrectly created elections.
US-12	Admin - Election Management	View Admin Dashboard Statistics	As an Admin, I want to see a summary dashboard with key metrics, so that I can

			monitor the overall health and activity of the voting system.
US-13	Voter - Casting a Vote	View Available Elections	As a Voter, I want to see a list of elections that are currently active, so that I know which elections I can participate in.
US-14	Voter - Casting a Vote	Check If I Have Already Voted	As a Voter, I want the system to automatically check if I have already voted in an election, so that I don't accidentally try to vote again.
US-15	Voter - Casting a Vote	Select a Candidate and Cast Vote	As a Voter, I want to select a candidate and submit my vote on the Blockchain, so that my choice is recorded transparently and immutably.
US-16	Voter - Casting a Vote	View My Vote Receipt After Voting	As a Voter, I want to see a receipt with my Transaction Hash and Vote Hash after voting, so that I can verify my vote was recorded on the Blockchain.
US-17	Voter - Casting a Vote	Be Protected from Double Voting	As a system, I want to prevent any wallet address from voting more than once in the same election, so that election integrity is guaranteed at the Blockchain level.
US-18	Transparency & Verification	View Live Election Results	As any user, I want to view the current vote counts for each candidate in real time, so that I can follow the progress of an election.
US-19	Transparency & Verification	View Winner After Election Ends	As any user, I want to see the winning candidate highlighted after an election is finalized, so that results are clear and publicly announced.

US-20	Transparency & Verification	View My Personal Transaction History	As a Voter, I want to see all my past votes in one place, so that I can track my participation history and verify my receipts at any time.
US-21	Transparency & Verification	Verify Any Transaction by Hash	As any user, I want to look up any transaction hash in the system, so that I can independently verify what happened in a specific blockchain transaction.
US-22	Transparency & Verification	View System Activity Log	As an Admin, I want to see a unified activity log of all system transactions, so that I can audit and monitor everything that has happened across all elections.

Table 3: User Stories

3.4 Business Process Model and Notation

The process starts with the Admin, who sets up the election, registers candidates, waits until the voting period begins, and then opens the voting process. During the voting period, the Admin monitors voting progress and checks whether the voting period is over. If the voting period has not ended, the system continues monitoring. If it has ended, the Admin closes voting, locks the ballot box, finalizes the vote count, and publishes the results.

In the Voter lane, the voter begins after voting is opened. The voter first authenticates voting eligibility, reviews candidate information, and decides whether to vote. If the voter has not made a decision, they return to reviewing candidate information. If a decision has been made, the voter casts a vote and receives a voting receipt. After the Admin publishes the results, the voter can view and verify the results before the process ends.

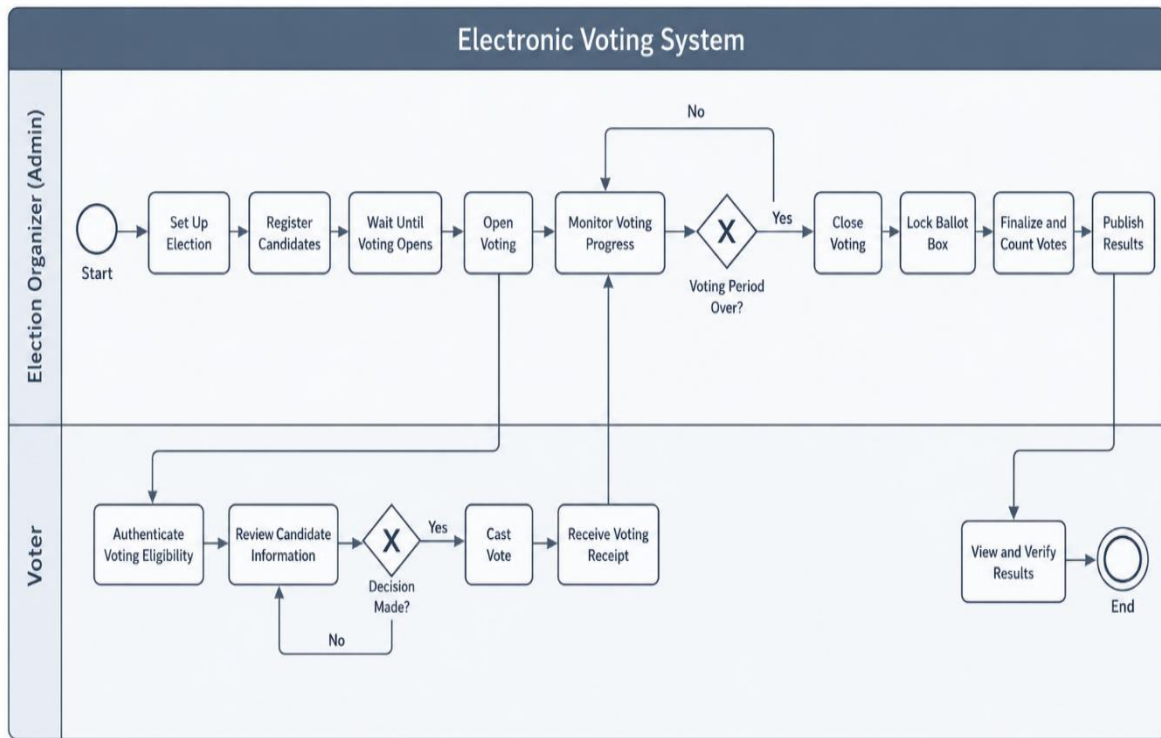


Figure 2: Business Process Model and Notation Diagram

3.5 Functional Requirements

FR Code	Function Name	Scenario	Role
FR-01	Account Registration	Given: The user is on the registration page. When: The user enters a username, email, and password, then submits the registration form. Then: The system creates a new account, securely stores the password, and assigns the default role as Voter.	Voter
FR-02	Login	Given: The user has an existing account. When: The user enters a valid email or username and password. Then: The system authenticates the user and creates a valid working session.	Admin / Voter
FR-03	Auto-Session Maintenance	Given: The user has previously logged in and the session is still valid.	Admin / Voter

		When: The user reloads or reopens the application. Then: The system automatically restores the user session without requiring a new login.	
FR-04	Logout	Given: The user is currently logged in. When: The user selects the logout option. Then: The system terminates the current session, disconnects the connected wallet, and redirects the user to the login page.	Admin / Voter
FR-05	Link MetaMask Wallet	Given: The user is logged in to the system. When: The user connects a MetaMask wallet. Then: The system synchronizes and links the current wallet address to the user account.	Admin / Voter
FR-06	Auto-Wallet Synchronization	Given: The user has already connected a wallet to the system. When: The user switches to another wallet account. Then: The system automatically updates the linked wallet address in the user account.	Admin / Voter
FR-07	Blockchain Network Verification	Given: The user attempts to connect a wallet. When: The connected wallet is not on the required blockchain network. Then: The system displays a warning and provides an option to switch to the required network.	Admin / Voter
FR-08	Create Election	Given: The Admin is authenticated and has access to election management. When: The Admin enters the election name, description, start date/time, and end date/time, then submits the request. Then: The system creates a new election and	Admin

		stores the election metadata together with its transaction record.	
FR-09	View Election List	Given: The Admin is on the election management page. When: The Admin opens the election list. Then: The system displays all created elections with their current status, number of candidates, and total votes cast.	Admin
FR-10	Filter Elections by Status	Given: The Admin is viewing the election list. When: The Admin selects a status filter such as All, Pending, Ongoing, Ended, or Finalized. Then: The system displays only the elections that match the selected status.	Admin
FR-11	Add Candidate	Given: The Admin has selected an existing election. When: The Admin enters the candidate's name and description, then submits the form. Then: The system adds the candidate to the selected election and stores the candidate record.	Admin
FR-12	Finalize Election	Given: The Admin is authorized and the election is ready to be finalized. When: The Admin selects the finalize action. Then: The system locks the election results, records the finalization transaction, and updates the election status to finalized.	Admin
FR-13	Delete Election	Given: The Admin selects an election that is still pending and has no recorded votes. When: The Admin confirms the delete action. Then: The system removes the selected election from the system.	Admin

FR-14	View Election Details	Given: The Admin selects a specific election. When: The election detail page is opened. Then: The system displays the candidate list, vote counts, transaction hash, and temporary result chart.	Admin
FR-15	View Election List	Given: The Voter is logged in. When: The Voter opens the election page. Then: The system displays all active elections with overview information and remaining voting time.	Voter
FR-16	Check Voting Status	Given: The Voter enters a voting page. When: The page is loaded. Then: The system checks whether the current wallet has already voted in the selected election.	Voter
FR-17	Select Candidate	Given: The Voter is viewing the candidate list. When: The Voter selects a preferred candidate. Then: The system visually highlights the selected candidate.	Voter
FR-18	Confirmation Before Voting	Given: The Voter has selected a candidate. When: The Voter proceeds to submit the vote. Then: The system displays a final confirmation message showing the selected candidate and warning that the action cannot be undone.	Voter
FR-19	Execute On-Chain Vote	Given: The Voter has confirmed the selected candidate and is eligible to vote. When: The Voter signs and submits the voting transaction. Then: The system executes the vote, records it on-chain, and waits for transaction confirmation.	Voter

FR-20	Record Vote Receipt	Given: The voting transaction has been successfully confirmed. When: The voting process is completed. Then: The system stores the transaction hash, anonymous vote hash, and block number as the voting receipt.	Voter
FR-21	Display Voting Results	Given: The Voter has successfully cast a vote. When: The success screen is displayed. Then: The system shows the transaction hash, vote hash, and block number with options to copy or view transaction details.	Voter
FR-22	Prevent Double Voting	Given: The Voter has already voted in the selected election. When: The Voter attempts to vote again. Then: The system rejects the voting action and displays a message indicating that the voter has already voted.	Voter
FR-23	Incorrect Network Protection	Given: The Voter is connected to an unsupported blockchain network. When: The Voter opens the voting page or attempts to vote. Then: The system disables the voting function and displays a warning with an option to switch to the required network.	Voter
FR-24	View Live Results	Given: An election is still in progress. When: A user opens or refreshes the results page. Then: The system displays the latest vote counts for each candidate.	Admin / Voter

FR-25	Results Visualization Charts	Given: Election result data is available. When: A user views the results page. Then: The system displays the results using visual charts showing vote counts and percentages.	Admin / Voter
FR-26	Winner Banner	Given: An election has ended or has been finalized. When: A user views the election results. Then: The system identifies the candidate with the highest number of votes and displays the winner banner.	Admin / Voter
FR-27	Personal Transaction History	Given: The Voter has previously cast one or more votes. When: The Voter opens the transaction history page. Then: The system displays the Voter's personal voting transaction history and supports searching by transaction hash or election ID.	Voter
FR-28	Transaction Lookup	Given: A user has a transaction hash. When: The user enters the transaction hash into the search field. Then: The system returns the related transaction information, including transaction type, timestamp, and associated election.	Admin / Voter
FR-29	System Transaction Table	Given: The Admin is viewing the dashboard. When: Recent system transactions are available. Then: The system displays a table of recent transactions related to election creation, candidate addition, voting, and finalization.	Admin

FR-30	Etherscan Integration	Given: A transaction hash is displayed in the system. When: The user clicks the transaction hash link. Then: The system redirects the user to the corresponding transaction detail page on the blockchain explorer.	Admin / Voter
FR-31	Copy Transaction Data	Given: A transaction hash or vote hash is displayed. When: The user clicks the copy button. Then: The system copies the selected cryptographic value to the clipboard and displays a confirmation notification.	Admin / Voter
FR-32	Overview Statistics	Given: The Admin opens the dashboard. When: Dashboard data is loaded. Then: The system displays key statistics including total elections, active elections, total recorded votes, and total registered users.	Admin
FR-33	Vote Comparison Chart	Given: Recent election data is available. When: The Admin views the dashboard. Then: The system displays a chart comparing the total votes across recent elections.	Admin
FR-34	Recent Elections List	Given: Elections have been created in the system. When: The Admin opens the dashboard. Then: The system displays up to five recent elections with their current status, turnout count, and vote metrics.	Admin
FR-35	Quick Actions	Given: The Admin is on the dashboard. When: The Admin selects a quick action button.	Admin

		Then: The system navigates the Admin directly to the selected module, such as creating an election, managing voters, or accessing the blockchain explorer.	
--	--	---	--

Table 4: Functional Requirements

3.6 Non-Functional Requirements

- NFR-01: Performance

ID	Requirement	Metric / Target
NFR-01.1	UI Response Time: UI pages (Dashboard, Election List, Results) must load and display data from the Backend API efficiently.	API response ≤ 500 ms under normal network conditions
NFR-01.2	Blockchain Transaction Latency : Wait time for transaction confirmation on the Sepolia network depends on the Ethereum network speed.	Transactions confirmed on average within 12–30 seconds after signing.
NFR-01.3	Database Query Speed: SQLite queries (fetching election lists, statistics, vote receipts) must execute quickly.	All DB queries complete in ≤ 100 ms
NFR-01.4	Dashboard Load Time: Admin Dashboard loads 3 APIs (getStats, getAll, getActivity) in parallel using Promise. all.	Entire Dashboard fully rendered in ≤ 2 seconds.
NFR-01.5	Concurrent Users: The system must serve multiple voters interacting simultaneously without data errors.	Supports a minimum of 100 concurrent users during the testing phase

Table 5: Non-Functional Requirements - Performance

- NFR-02: Security

ID	Requirement	Metric / Target
----	-------------	-----------------

NFR-02.1	Authentication: All sensitive API endpoints are protected by JWT Middleware. The system must reject any request missing or contains an invalid Token.	100% of sensitive read/write APIs require a valid Token; returns HTTP 401 if missing
NFR-02.2	Password Hashing: User passwords must be hashed before being saved to the database; they must not be stored as plaintext.	Use bcryptjs with salt rounds ≥ 10
NFR-02.3	Private Key Protection: The Admin/Deployer's Private Key must not be hardcoded in the source code and must not be committed to the version control system (Git).	Private Key must be placed in the .env file, and this file must be included in .gitignore
NFR-02.4	Double Voting Prevention: The system must absolutely prevent a single wallet from voting twice in the same election.	Enforced at the Smart Contract level using the hasVoted mapping; cannot be bypassed from Frontend or Backend.
NFR-02.5	Role-Based Access Control (RBAC): Admin pages and APIs must be inaccessible to accounts with the voter role.	AdminRoute component blocks 100% of unauthorized access; Admin APIs require req.user.role = admin
NFR-02.6	CORS Protection: The backend only accepts HTTP requests from allowed origins.	Only accepts requests from the allowedOrigins list explicitly configured in app.js
NFR-02.7	Smart Contract Owner Control: Only the wallet that deployed the contract (owner) can call administrative functions (createElection, addCandidate, finalizeElection).	The onlyOwner modifier is checked on every admin function in the Smart Contract.
NFR-02.8	Transaction Immutability: Any transaction recorded on the Blockchain cannot be deleted	Immutability is guaranteed by the Ethereum Blockchain architecture

	or modified by any party, including the Admin.	
--	--	--

Table 6: Non-Functional Requirements - Security

- NFR-03: Reliability & Availability

ID	Requirement	Metric / Target
NFR-03.1	System Uptime: The Backend and Frontend must maintain continuous operation while elections are active.	Target uptime $\geq 99\%$ during active election periods
NFR-03.2	Graceful Error Handling: All errors (network, MetaMask, Server) must be handled and display user-friendly messages.	No errors are displayed as stack traces or white/black screens to end-users
NFR-03.3	Transaction Failure Recovery: If a Blockchain transaction fails (low gas, rejected), the system must clearly notify the user and allow them to retry.	Display meaningful error messages within ≤ 3 seconds after a transaction failure.
NFR-03.4	Data Consistency: Data in SQLite (Database) must always remain consistent with the actual state on the Blockchain.	All on-chain changes are synced to the Database immediately after the transaction is mined.
NFR-03.5	Database Integrity: The Database must be protected against duplicate records.	The tx_hash is declared as UNIQUE NOT NULL in the vote_receipts table.

Table 7: Non-Functional Requirements - Reliability and Availability

- NFR-04: Usability

ID	Requirement	Metric / Target
NFR-04.1	Intuitive Navigation: New users can find and execute the voting process without special instructions.	New users complete their first voting action in ≤ 5 minutes.

NFR-04.2	Real-time Feedback: All user interactions (button clicks, form submissions, signing transactions) must have instant UI feedback.	Use toast notifications (react-hot-toast) and loading spinners for feedback within $\leq 300\text{ms}$
NFR-04.3	Visual Status Indicators: The status of each election and MetaMask wallet must be clearly indicated using colours and labels.	Distinct colour badges for: Pending (yellow) / Active (green) / Ended (gray) / Finalized (purple)
NFR-04.4	Empty State Handling: When there is no data, the system must show friendly messages instead of blank tables.	All list pages have an empty state component with actionable guidance messages.
NFR-04.5	Language: The entire user interface uses Vietnamese to suit the target audience.	100% of labels, messages, and instructions on the UI are in Vietnamese
NFR-04.6	Dark Mode: The interface uses a modern, system-wide, consistent Dark Mode.	Design system with CSS variables defined in index.css; consistent across all pages

Table 8: Non-Functional Requirements - Usability

- NFR-05: Compatibility

ID	Requirement	Metric / Target
NFR-05.1	Browser Support: The application must fully function on popular browsers that support MetaMask.	Full support for: Google Chrome ≥ 90 , Microsoft Edge ≥ 90 , Firefox ≥ 88
NFR-05.2	MetaMask Dependency: All Blockchain interaction features require the MetaMask Extension to be installed.	If MetaMask is not installed, the system prompts the user to install it instead of crashing.
NFR-05.3	Network Compatibility: The system operates on the Sepolia Testnet with Chain ID 11155111.	Smart Contract deployed on Sepolia; System rejects transactions if the wallet is on another network

NFR-05.4	Responsive Design: The interface must render correctly and be usable on both Desktop and Mobile.	Breakpoint at 768px: Sidebar hides, grid layout collapses to 1 column, padding adjusts
NFR-05.5	Node.js Compatibility: The Backend runs stably on common LTS versions of Node.js.	Requires Node.js $\geq 18.x$ LTS

Table 9: Non-Functional Requirements - Compatibility

- NFR-06: Maintainability

ID	Requirement	Metric / Target
NFR-06.1	Code Separation: Source code must be clearly organized following the Separation of Concerns principle.	Frontend: pages/, components/, api/, context/ separated; Backend: routes/, controllers/, middlewares/, config/
NFR-06.2	Database Migration: There is a mechanism to update the database schema without losing existing data.	The migrate.js script executes ALTER TABLE commands to add new columns to populated tables
NFR-06.3	Environment Configuration: All environment variables must be placed in .env and not hardcoded in the source code.	Use dotenv for Backend; VITE_* prefix for Frontend. An .env.example template is provided for each module
NFR-06.4	API Consistency: All API endpoints must return a consistent JSON response structure.	All responses follow the format: { success: boolean, message?: string, data?: object }
NFR-06.5	Contract ABI Synchronization: When the Smart Contract is redeployed, the ABI file must be updated simultaneously for both Frontend and Backend.	The deploy.js script automatically copies Voting.json to frontend/src/contracts/ and backend/src/contracts/

Table 10: Non-Functional Requirements - Maintainability

- NFR-07: Transparency & Auditability

ID	Requirement	Metric / Target
NFR-07.1	On-chain Traceability: All critical actions (creating elections, voting, finalizing) must be recorded on the Blockchain with unique transaction hashes.	100% of major transactions have their tx_hash saved and displayed in the system
NFR-07.2	Vote Anonymity: Voting results must ensure anonymity: the system records that a wallet has voted but does not reveal which candidate was chosen.	Vote Hash is generated using keccak256: a one-way function that cannot be reversed
NFR-07.3	External Verification: Anyone can independently verify any transaction via Etherscan without logging into the system.	Every tx_hash in the system has a direct link to sepolia.etherscan.io
NFR-07.4	Result Integrity: Election results are drawn directly from on-chain vote counts and cannot be manipulated by the Admin via the Database.	Vote counts are stored in Smart Contract storage (voteCount): completely independent of SQLite

Table 11: Non-Functional Requirements - Transparency and Auditability

- NFR-08: Scalability

ID	Requirement	Metric / Target
NFR-08.1	Election Capacity: The system must support a large number of elections without logical limitations.	The Smart Contract uses uint256 for electionCount: supporting up to $2^{256} - 1$ elections
NFR-08.2	Candidate Capacity: The number of candidates in each election is not hard-limited in the Smart Contract.	No hardcoded limits; practical limits depend on the Sepolia block gas limit
NFR-08.3	Database Growth: SQLite is suitable for testing and learning phases; can be	Controller/Query layer architecture is designed to swap DB engines without changing Controller logic

	migrated to PostgreSQL when scaling to production.	
NFR-08.4	Multi-chain Readiness: The Frontend architecture is designed to easily support additional blockchain networks in the future.	Chain ID is configured via the VITE_CHAIN_ID environment variable; not hardcoded into business logic

Table 12: Non-Functional Requirements - Scalability

3.7 Use Case Diagram

The Use Case Diagram of the Decentralized Voting System shows the main system functions and how three actors interact with them: Guest, Voter, and Admin. The Guest can only register or log in, while the Voter can connect a MetaMask wallet, view elections, cast a vote, receive a receipt, verify the vote, and review voting history.

The Admin has the highest permission level, including creating elections, adding candidates, finalizing elections, deleting pending elections, viewing dashboard statistics, managing users, and checking blockchain transactions. The use cases are grouped into five main areas: Authentication, Election Information, Voting, Election Management, and Administration.

The diagram also uses «include» relationships to show required actions. For example, voting and key Admin operations require connecting a MetaMask wallet before blockchain transactions can be signed.

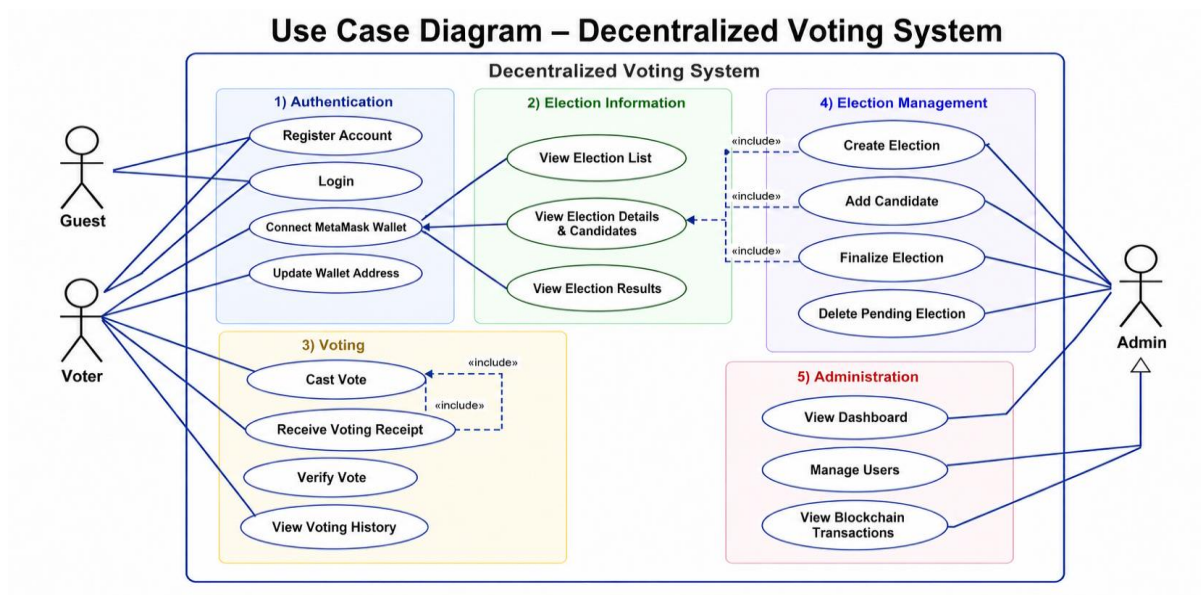


Figure 3: Use Case Diagram of the Decentralized Voting System

- Submit Data Process:

This use case diagram describes the voting process of the Voter in the Voting System. First, the voter can view the list of open elections and review the candidate list before making a decision. When the voter performs the Vote for Candidate use case, the system must check whether the voter has already voted, require the voter to connect a MetaMask wallet, sign the transaction for confirmation, and then generate a voting receipt in the form of a tx_hash.

The «include» relationships indicate mandatory steps in the voting process. These steps ensure that each voter can vote only once, the transaction is properly confirmed through the digital wallet, and the voter receives a receipt for later verification. The «extend» relationship between viewing the candidate list and viewing open elections shows that candidate information is usually accessed after the voter selects a specific open election

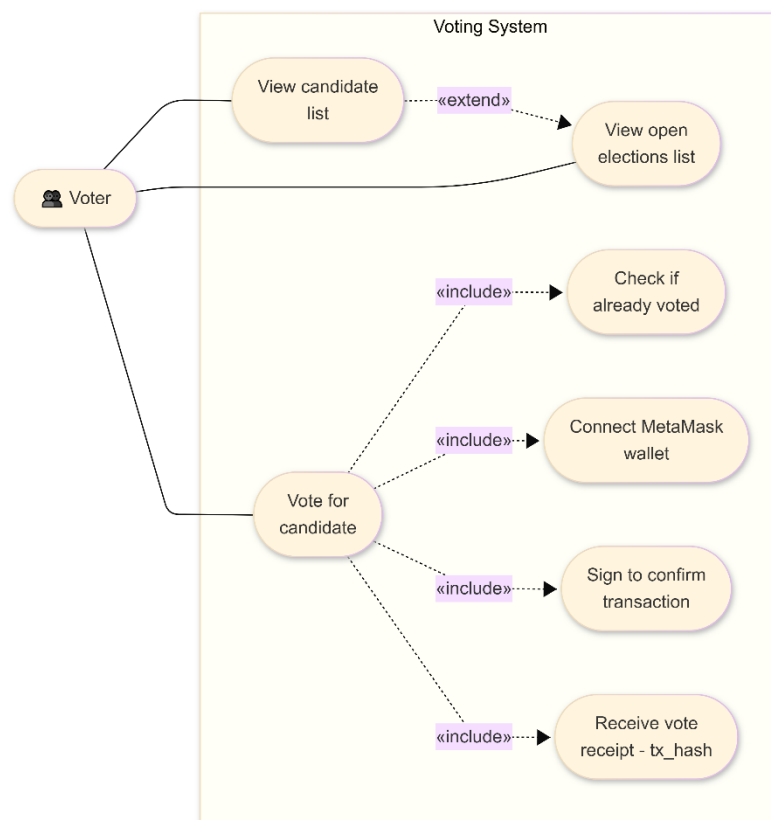


Figure 4: Submit Data Process Use Case Diagram

- Verify Data Process:

The Verify Data Process diagram describes the data verification process in the voting system. Both Voter and Admin can perform the Verify vote validity function to check the validity of a vote or a related transaction.

For the Voter, the system allows users to view their personal vote receipt and transaction details to confirm that their vote has been recorded. For the Admin, the system supports verification of the election creation transaction to ensure transparency in administrative data.

The Verify vote validity function must include the step Enter transaction hash (tx_hash) because the transaction hash is the main data used for lookup and verification. The «extend» relationships show that viewing transaction details, viewing personal vote receipts, and verifying election creation transactions are extended functions that support the main verification process.

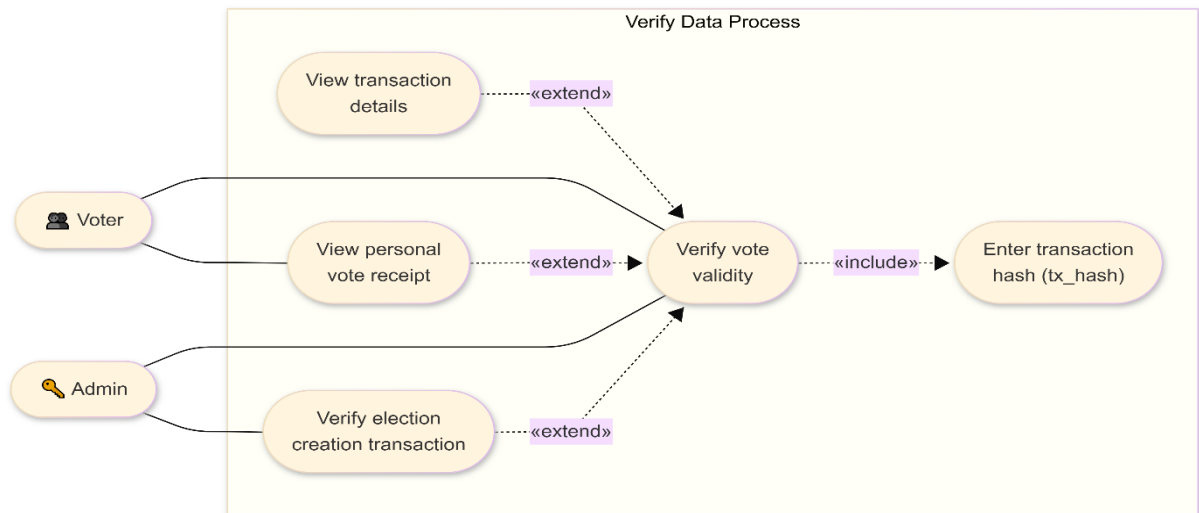


Figure 5: Verify Data Process Use Case Diagram

- Create Transaction Process:

This diagram describes the Create Transaction process performed by the Admin in the voting system. The Admin can create a new election on the blockchain, add candidates to an election, finalize an election, and delete an election while it is still pending.

The main use case is Create new election on Blockchain, which represents the process of recording election data on-chain. This action requires several mandatory steps, shown by the «include» relationships. The Admin must connect the MetaMask wallet, sign to confirm the on-chain transaction, and then synchronize the transaction data to the database.

Other actions such as Add candidate to election and End election (Finalize) also require wallet connection and transaction signing because they modify blockchain data. The Delete election while Pending function is more limited and is only used before the election starts.

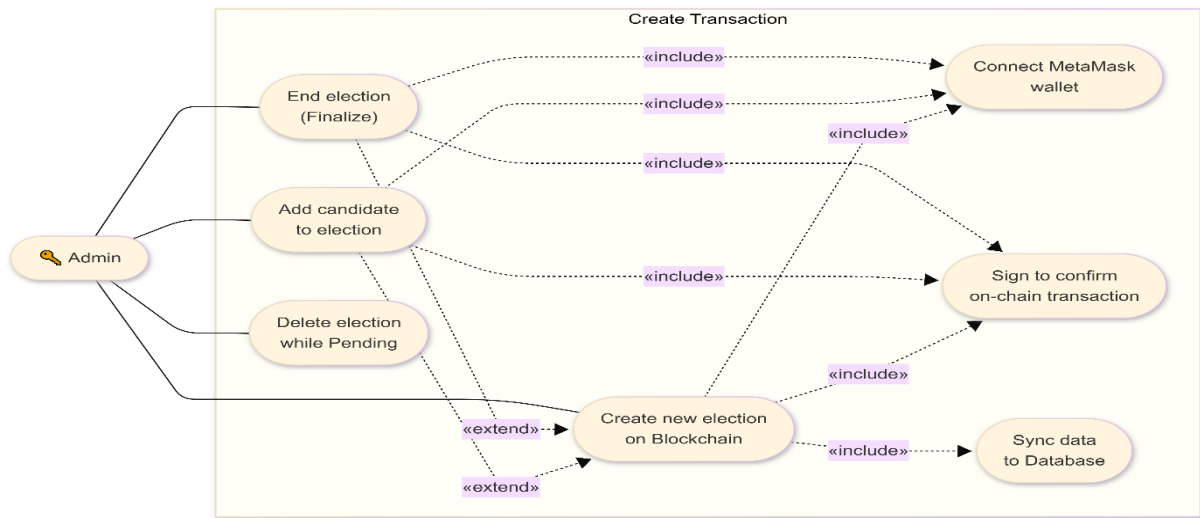


Figure 6: Create Transaction Process Use Case Diagram

- View History Process:

The View History diagram describes how voters and administrators access historical records and result information in the voting system. The Voter can view personal voting history, check cast vote details, filter history by election, and view election result charts. These functions help voters track their own voting activities and verify their participation after casting a vote.

The Admin has broader access to system-level information, including blockchain transaction history, overview statistics dashboard, and election result charts. These functions allow the Admin to monitor system activity, review voting records, and evaluate election outcomes.

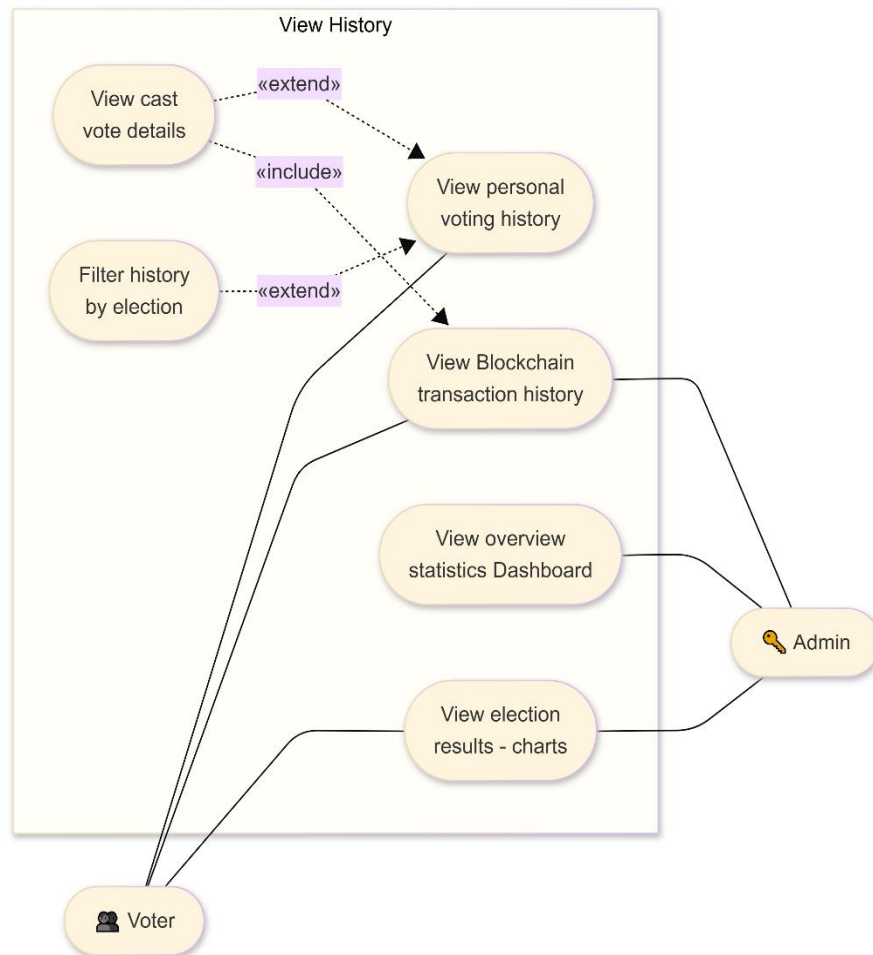


Figure 7: View History Process Use Case Diagram

- Use Case Distribution by Actor and Execution Layer:

The chart shows the distribution of use cases based on two dimensions: actor type and data execution layer. The horizontal axis separates actions by actor, from Admin-only actions on the left to Voter-only actions on the right. The vertical axis separates actions by where they are mainly processed: on-chain blockchain actions at the top and off-chain database actions at the bottom.

Admin on-chain actions include Create Election, Add Candidate, and Finalize Election, because these functions modify important election data on the blockchain. Voter on-chain action mainly includes Cast Vote, which must be recorded securely and transparently. Shared actions such as Connect MetaMask and Search Transaction are placed near the center because both Admin and Voter may use them.

Most viewing-related features, such as View Dashboard Stats, View System Activity, View Active Elections, View Vote History, and View Results, are placed in the off-chain area

because they mainly retrieve and display stored data. Overall, the diagram shows that critical election actions are handled on-chain, while supporting and viewing functions are mainly handled off-chain for better usability and performance.

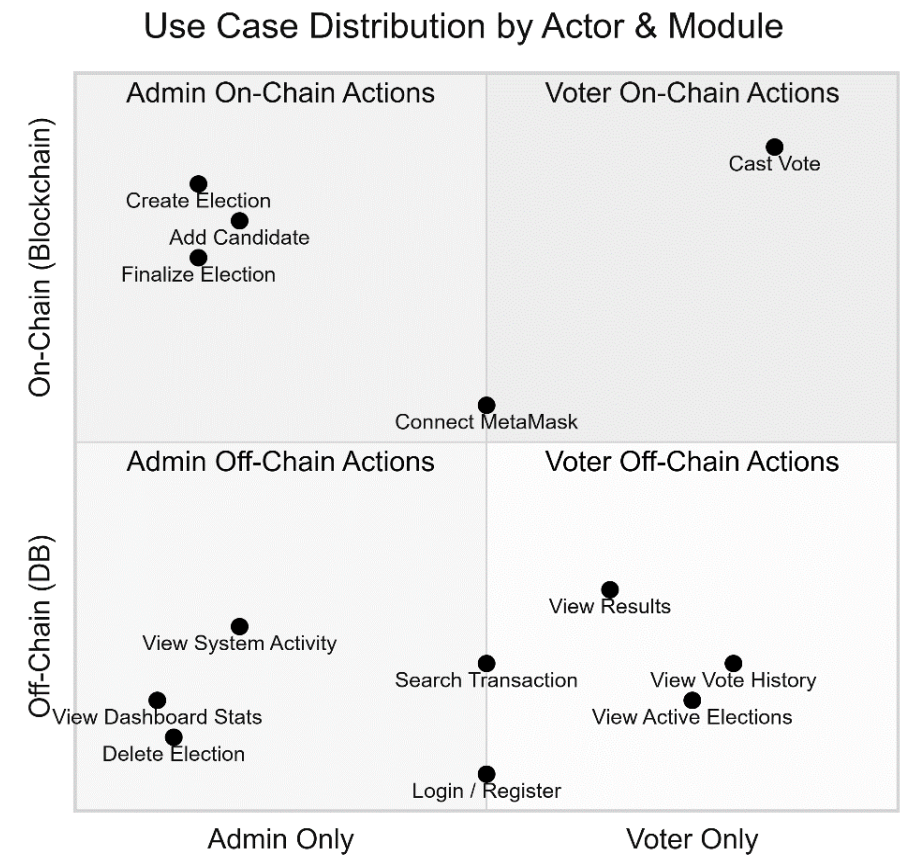


Figure 8: Use Case Distribution by Actor and Execution Layer

3.8 Main Workflow

The main workflow of the decentralized voting system consists of five key phases: login and authentication, election preparation, voter participation, voting, and result publication. First, users access the system by logging in with their account credentials. After successful authentication, they connect their digital wallet, which is used to confirm their identity and securely sign important election-related actions. This ensures that only valid users can participate in the voting process.

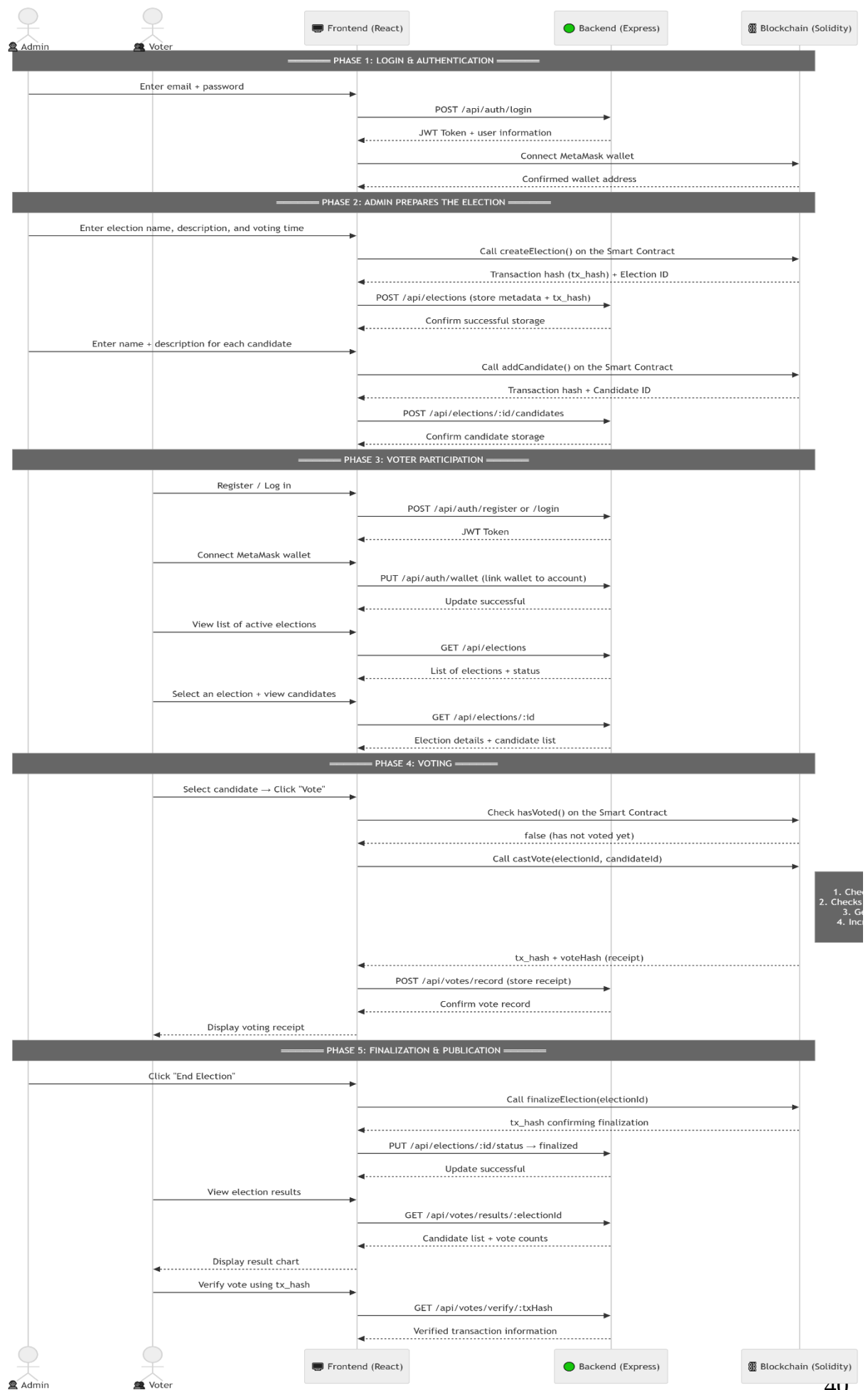
In the election preparation phase, the Admin creates a new election by entering essential information such as the election name, description, start time, and end time. The Admin also registers the list of candidates who will participate in the election. Election and candidate data are stored both on-chain and off-chain. On-chain storage ensures transparency, immutability,

and auditability, while off-chain storage helps the system retrieve and display information more quickly for users.

After the election becomes active, voters can access the list of available elections. They can review election details, read candidate information, and decide which candidate they want to vote for. Before a vote is submitted, the system verifies the voter's eligibility and checks whether the voter has already voted in the same election. This step is important to prevent duplicate voting and maintain the fairness of the election.

Once the voter selects a candidate and confirms the action, the vote is recorded as a secure transaction. The system then generates an anonymous voting receipt, allowing the voter to verify that their vote has been recorded without revealing which candidate they selected. This helps protect voter privacy while still maintaining transparency and trust in the voting process.

Finally, when the voting period ends, the Admin finalizes the election and publishes the official results. Voters can view the result chart, verify their vote using the transaction hash, and review their personal voting history. Overall, this workflow provides a complete voting process that is secure, transparent, auditable, privacy-preserving, and easy to use from the beginning of the election to the final publication of results.



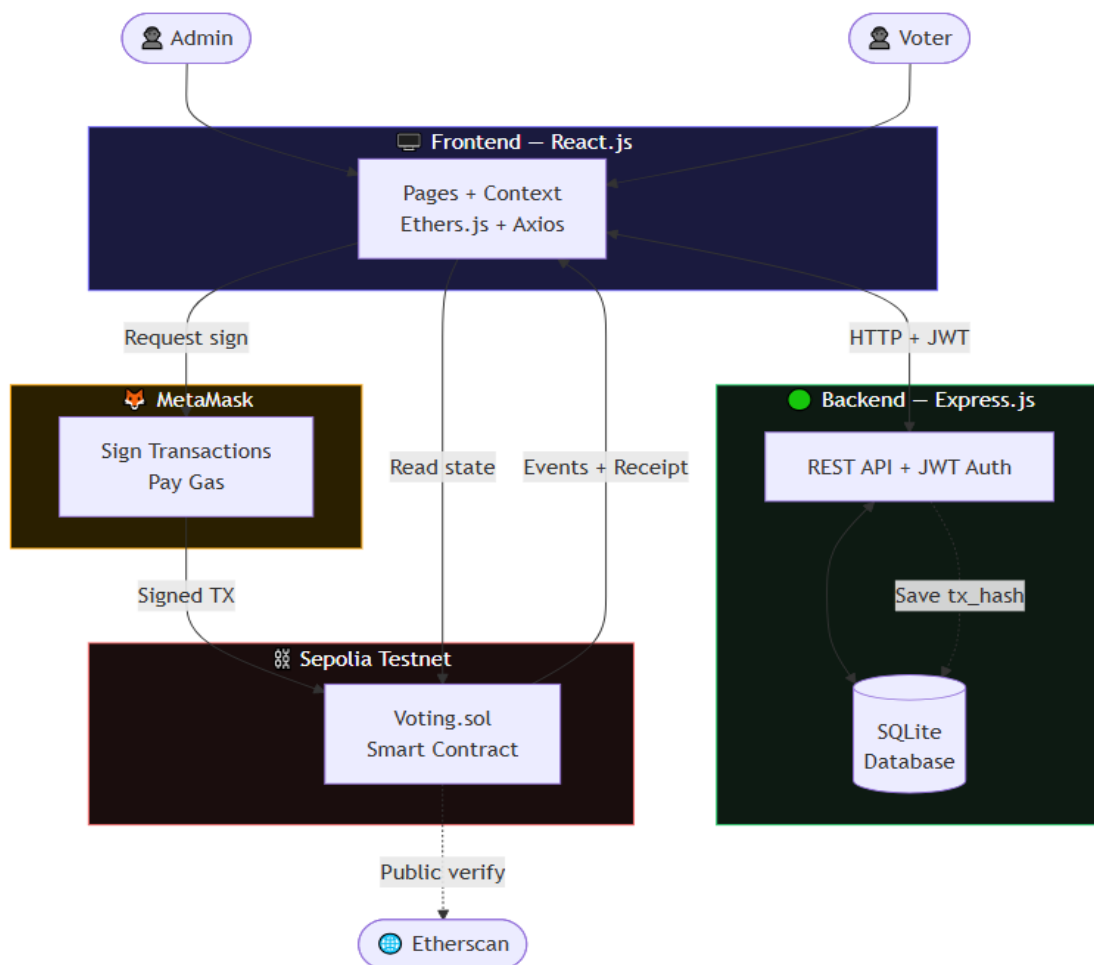
1. Check
2. Check
3. Get
4. Inc

Figure 9: Main Workflow of the Decentralized Voting System

3.9 System Design

3.9.1 Overall Architecture

The VoteChain decentralized voting system operates through a seamless coordination of four core components to ensure absolute transparency and security. At the user interaction layer, the React.js Frontend acts as the central coordinator for all data flows, delivering a smooth and intuitive experience for both voters and administrators. When blockchain interactions are required, the MetaMask wallet takes charge of signing and authenticating transactions, ensuring that the user's Private Key is never exposed to the application. In the background, the Express.js/Node.js Backend processes auxiliary logic and securely stores election metadata within a local SQLite database to optimize data retrieval speeds. Finally, all core voting processes are executed directly by the Smart Contract on the Ethereum Sepolia Testnet, guaranteeing that the election rules and final ballot results remain completely immutable, transparent, and tamper-proof.



3.9.2 Database Design

The relationships in the ERD show how the main data entities of VoteChain are connected to support the voting process. The USERS table has a one-to-many relationship with the ELECTIONS table, meaning that one Admin user can create multiple elections, while each election is created by only one user through the created_by foreign key.

The ELECTIONS table is the central entity of the database because most other entities depend on it. One election can have many CANDIDATES, many AUTHORIZED_VOTERS, and many VOTE_RECEIPTS. This reflects the real voting process, where each election contains a candidate list, a group of eligible voters, and multiple voting records after voters cast their votes.

The relationship between USERS and AUTHORIZED_VOTERS shows which users are allowed to participate in specific elections. Each authorized voter record links a user account, a wallet address, and an election. This helps the system manage voter eligibility and ensure that only valid voters can vote.

The relationship between ELECTIONS and CANDIDATES shows that candidates are always associated with a specific election. A candidate cannot exist independently without belonging to an election. The on_chain_id also connects the off-chain candidate record with the corresponding candidate stored on the blockchain.

Finally, the relationship between ELECTIONS and VOTE_RECEIPTS shows that each vote receipt belongs to a specific election. These receipts store transaction hashes and anonymous vote hashes, allowing users to verify voting participation while maintaining voter privacy.

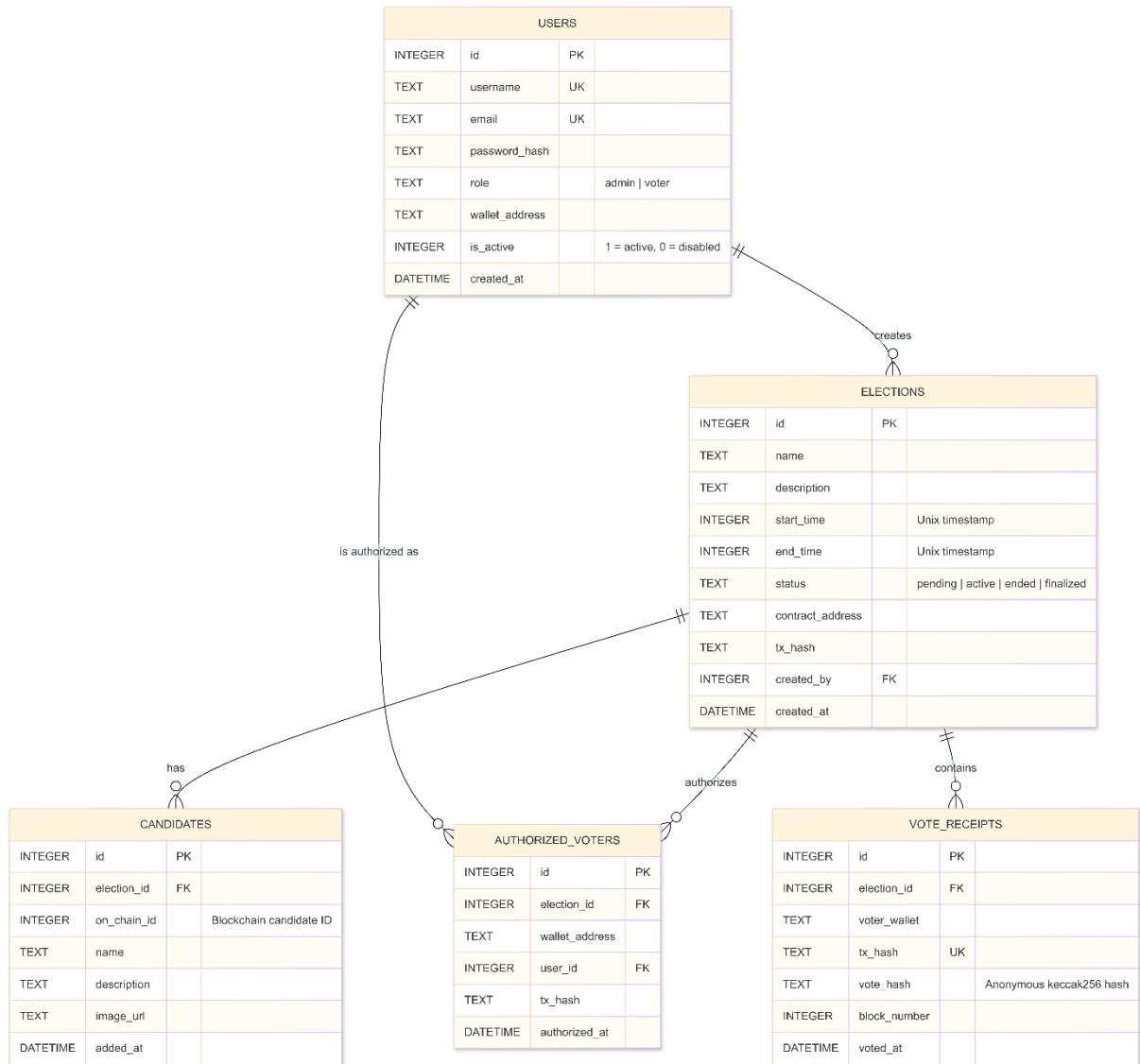


Figure 11: Entity Relationship Diagram

3.9.3 Data Design

Data Entity	Main Data Fields	Description	Purpose in the System
Users	id, username, email, password_hash, role, wallet_address, is_active, created_at	Stores all user account information in the system, including Admin and Voter accounts. Each user has a unique username/email, a secured password hash, a role, an	Used for user authentication, role-based access control, account management, wallet linking, and identifying whether a

		optional wallet address, and an account status.	user is allowed to access the system.
Elections	id, name, description, start_time, end_time, status, contract_address, tx_hash, created_by, created_at	Stores election metadata, including election title, description, voting period, current status, related blockchain transaction hash, contract address, and the Admin who created the election.	Used to manage the full election lifecycle, from creation, pending state, active voting period, ended state, to finalization and result publication.
Candidates	id, election_id, on_chain_id, name, description, image_url, added_at	Stores candidate information for each election. Each candidate belongs to one election and is mapped to a corresponding candidate ID on the blockchain through on_chain_id.	Used to display candidate lists, show candidate details to voters, and connect off-chain candidate profiles with on-chain voting records.
Authorized Voters	id, election_id, wallet_address, user_id, tx_hash, authorized_at	Stores information about voters who are allowed to participate in a specific election. Each record links an election, a user account, and a wallet address.	Used to control voter eligibility, ensure that only approved voters can participate in selected elections, and track authorization transactions.
Vote Receipts	id, election_id, voter_wallet, tx_hash, vote_hash, block_number, voted_at	Stores voting receipt information after a voter successfully casts a vote. Each receipt contains the election ID, voter wallet, transaction hash, anonymous vote hash,	Used for vote verification, voting history review, transaction tracking, and providing proof that a vote has been recorded without

		block number, and voting time.	exposing the selected candidate.
--	--	--------------------------------	----------------------------------

Table 13: Main Data Description

3.9.4 Smart Contract Design

- Candidate Data Structure

Attribute Name	Data Type	Description
id	uint256	The unique identifier of the candidate within an election.
name	string	The full name of the candidate.
description	string	A short description or profile information of the candidate.
voteCount	uint256	The current total number of votes received by the candidate.

Table 14: Candidate Data Structure

- Election Data Structure

Attribute Name	Data Type	Description
id	uint256	The unique identifier of the election.
name	string	The title or name of the election.
description	string	A detailed description of the purpose of the election.
startTime	uint256	The starting time of the election, stored as a Unix Timestamp.
endTime	uint256	The ending time of the election, stored as a Unix Timestamp.

exists	bool	Indicates whether the election exists on-chain and is used for validation.
finalized	bool	Indicates whether the election has been finalized and closed by the Admin.
candidateCount	uint256	The total number of candidates participating in the election.
voterCount	uint256	The total number of voters who have successfully cast their votes.

Table 15: Election Data Structure

- Main State Variables

These variables store data directly in the blockchain storage state.

Variable Name	Data Type	Visibility	Detailed Description
owner	address	public	Stores the wallet address of the Admin who deploys and manages the contract.
electionCount	uint256	public	Counts the total number of elections created in the system.
elections	mapping(uint256 => Election)	public	Maps an election ID to the detailed information of that election.
candidates	mapping(uint256 => mapping(uint256 => Candidate))	public	A nested mapping that stores the candidate list using the structure: electionId => candidateId => Candidate.
hasVoted	mapping(uint256 => mapping(address => bool))	public	A nested mapping that marks whether a voter has voted: electionId => voterAddress => true/false.

voteReceipts	mapping(uint256 => mapping(address => bytes32))	public	Stores anonymous encrypted voting receipts using the structure: electionId => voterAddress => voteHash.
--------------	---	--------	---

Table 16: Main State Variables

- Modifiers

Modifiers are condition filters used to validate requirements before a function is executed.

Modifier Name	Parameter	Constraint Description
onlyOwner	None	Requires the function caller address (msg.sender) to match the owner wallet address.
electionExists	uint256 _electionId	Verifies that the election with the given ID exists on the blockchain (exists == true).
electionActive	uint256 _electionId	Requires the election to be active, not ended by time, and not marked as finalized.
notVoted	uint256 _electionId	Ensures that the caller's wallet address has not voted in this election before.

Table 17: Smart Contract Modifiers

- Member Functions

These are the main business functions implemented in the smart contract.

Function Name	Parameters	Return Value	Visibility	Business Description
createElection	_name, _description, _startTime, _endTime	uint256 - new election ID	external onlyOwner	Creates a new election on-chain. This function can only be executed by the Admin.

addCandidate	_electionId, _name, _description	uint256 - candidate ID	external onlyOwner	Adds a new candidate to an election before voting starts.
finalizeElection	_electionId	None	external onlyOwner	Officially closes and finalizes the election. This function can only be executed by the Admin.
castVote	_electionId, _candidateId	None	external notVoted	Allows a voter to cast a vote for a candidate. The system automatically generates an anonymous hashed receipt for privacy.
getResults	_electionId	names[], descriptions[], voteCounts[]	external view	Returns the list of candidates and their corresponding vote counts.
getVoteReceipt	_electionId, _voter	bytes32 - hash value	external view	Retrieves the encrypted voting receipt so that voters can verify transparency.
getElectionStatus	_electionId	string - pending, active, ended, finalized	external view	Analyzes the current block time and returns the status of the election.

Table 18: Smart Contract Member Functions

- Events

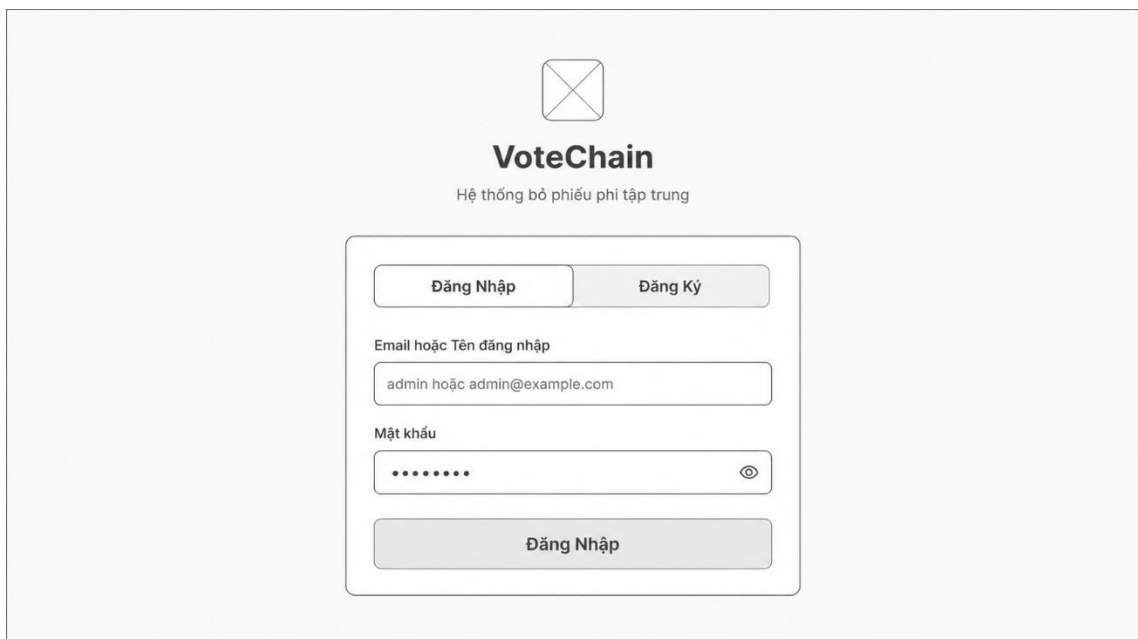
Events are logs recorded in the EVM, allowing the frontend to listen to system status changes in real time.

Event Name	Recorded Parameters	Meaning and Trigger Time
ElectionCreated	electionId, name, startTime, endTime	Emitted immediately after the Admin successfully creates a new election.
CandidateAdded	electionId, candidateId, name	Emitted when a new candidate is added to an election.
VoterAuthorized	electionId, voter wallet address	Emitted when a voter is authorized to participate in an election.
VoteCast	electionId, voter, voteHash, timestamp	Emitted immediately after a voter successfully casts a vote. It contains the anonymous hashed receipt.
ElectionFinalized	electionId, timestamp	Emitted when the Admin officially closes the election.

Table 19: Smart Contract Events

3.9.5 User Interface Design

- Login/Register:



The image shows a user interface for a system called "VoteChain". At the top, there is a logo consisting of a square with an 'X' inside. Below the logo, the text "VoteChain" is displayed in a bold font, followed by the subtitle "Hệ thống bỏ phiếu phi tập trung" in a smaller font. The main part of the interface is a login/register form. At the top of this form, there are two buttons: "Đăng Nhập" (Login) and "Đăng Ký" (Register). Below these buttons, there is a label "Email hoặc Tên đăng nhập" (Email or Username) and a text input field containing the placeholder text "admin hoặc admin@example.com". Underneath this is a label "Mật khẩu" (Password) and a password input field with a toggle icon (an eye) on the right. At the bottom of the form, there is a large "Đăng Nhập" (Login) button.

Figure 12: Login/Register Interface Design

- **Dashboard:**

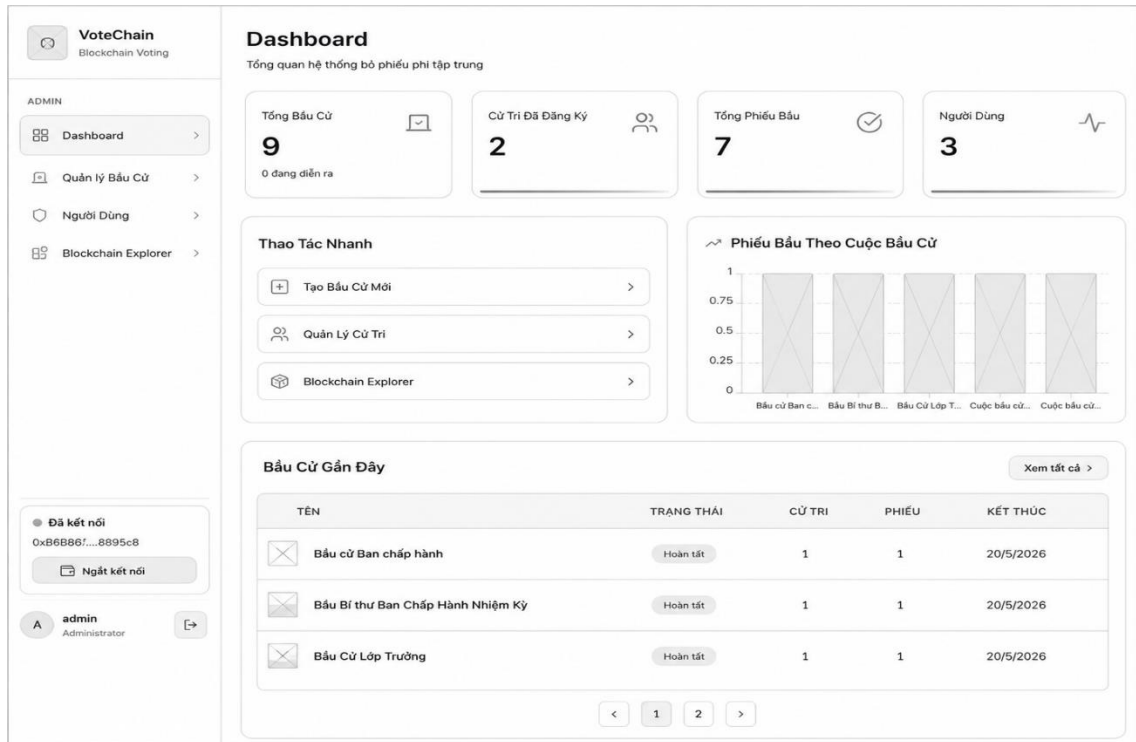


Figure 13: Admin Dashboard Interface Design

- **Create Election (Admin):**

The 'Quản Lý Bầu Cử' (Manage Elections) interface for an admin. It includes a sidebar with navigation links: Dashboard, Quản lý Bầu Cử, Người Dùng, and Blockchain Explorer. The main content area has a 'Tạo Bầu Cử' button and a search bar. Below is a list of elections with details like name, description, dates, and status. A modal form titled 'Tạo Bầu Cử Mới' is open, allowing the admin to create a new election. The form includes fields for 'Tên bầu cử', 'Mô tả', 'Ngày bắt đầu', 'Giờ bắt đầu', 'Ngày kết thúc', and 'Giờ kết thúc'.

Tạo Bầu Cử Mới

Tên bầu cử *

VD: Bầu cử Hội đồng sinh viên 2025

Mô tả

Mô tả chi tiết về cuộc bầu cử...

Ngày bắt đầu *

Giờ bắt đầu

dd/mm/yyyy 09:00 SA

Ngày kết thúc *

Giờ kết thúc

dd/mm/yyyy 06:00 CH

Hủy + Tạo Bầu Cử

Figure 14: Create Election Modal Interface Design

- Add Candidates (Admin):

The screenshot shows the 'Bầu cử ban chấp hành LCD Hoà Lạc' (LCD Hoà Lạc Executive Board Election) interface for an admin user. The left sidebar contains the 'ADMIN' menu with options: Dashboard, Quản lý Bầu Cử (selected), Người Dùng, and Blockchain Explorer. Below the menu is a section for 'Đã kết nối' (Connected) showing the address 0x068067...089548 and a 'Ngắt kết nối' (Disconnect) button. The user profile at the bottom shows 'admin' with a 'Logout' button. The main content area has a title 'Bầu cử ban chấp hành LCD Hoà Lạc' with a 'View Tx' button. Below the title is a 'Sắp diễn ra' (Upcoming) status and 'ID: 10'. The 'Thông tin' (Information) section displays the election name, start time '12:26:00 22/5/2026', and end time '12:50:00 22/5/2026'. To the right, a box shows 'Tổng Lượt Bầu' (Total Votes) as 0 and 'Giao Dịch Ghi Nhận' (Transaction Recorded). Below this, the 'Ứng Viên (0)' (Candidates) section shows 'Chưa có ứng viên nào' (No candidates yet) and a form to 'Thêm ứng viên mới' (Add new candidate) with fields for 'Tên ứng viên' (Candidate name) and 'Tên/biệt danh ứng viên' (Candidate name/alias), and a '+ Thêm' (Add) button. On the far right, the 'Cử Tri Đã Bầu' (Voters who have voted) section shows 'Chưa có ai bầu' (No one has voted yet).

Figure 15: Add Candidates and Election Detail Interface Design

- Vote (Voter)

The screenshot shows the 'Bầu cử ban chấp hành LCD Hoà Lạc' (LCD Hoà Lạc Executive Board Election) interface for a voter user. The left sidebar contains the 'MENU' with options: Bầu Cử (selected), Lịch sử phiếu bầu (Ballot history), and Tra Cứu Phiếu (Ballot lookup). Below the menu is a section for 'Đã kết nối' (Connected) showing the address 0xbb5018...b5d551 and a 'Ngắt kết nối' (Disconnect) button. The user profile at the bottom shows 'Khanh' with the role 'voter' and a 'Logout' button. The main content area has a title 'Bầu cử ban chấp hành LCD Hoà Lạc' and the subtitle 'Bầu cử ban chấp hành LCD Hoà Lạc'. Below the title is the section 'Chọn ứng viên của bạn' (Choose your candidate) with two options: '1 A' and '2 B', each with a radio button. At the bottom, there is a 'Bỏ Phiếu' (Cast ballot) button.

Figure 16: Voter Voting Interface Design

- View history (Voter)

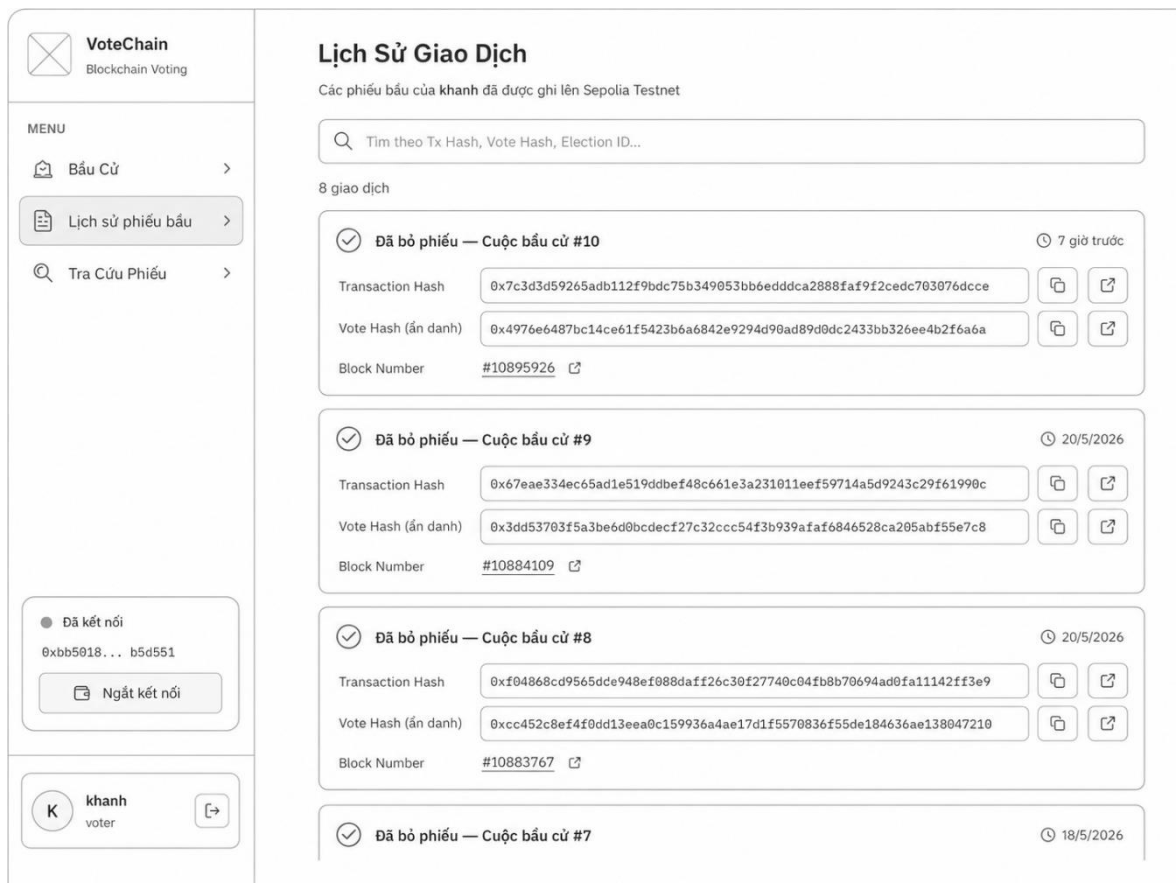


Figure 17: Vote History Interface Design

- Results:

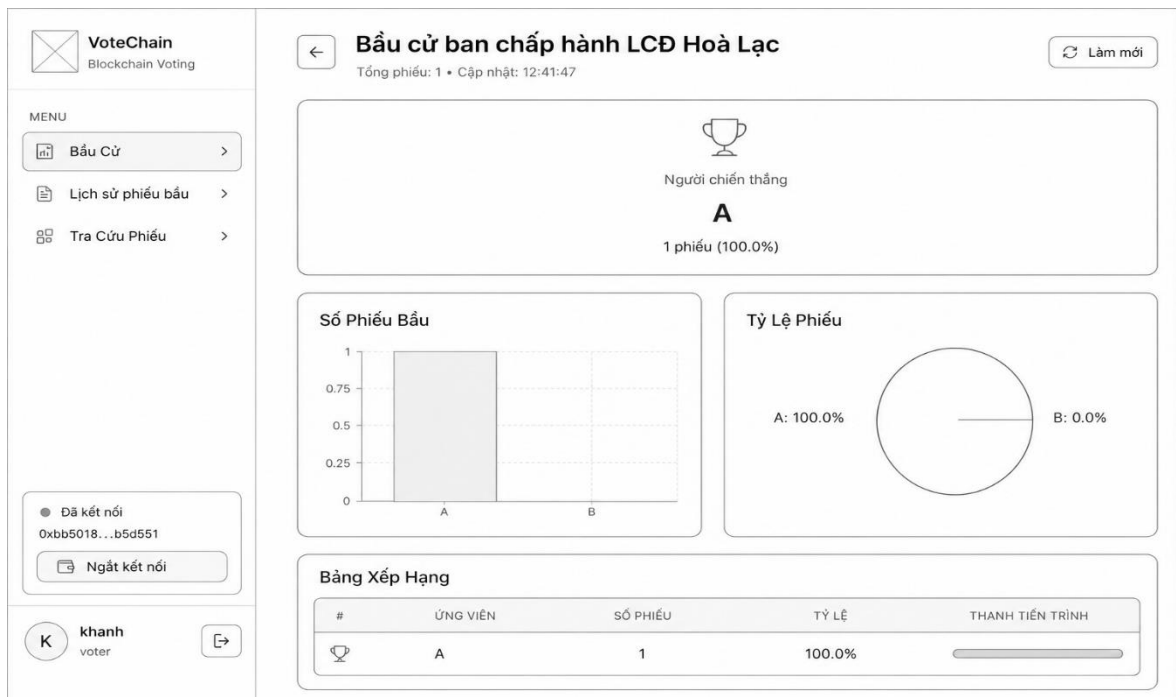


Figure 18: Election Results Interface Design

- *Blockchain Explorer*

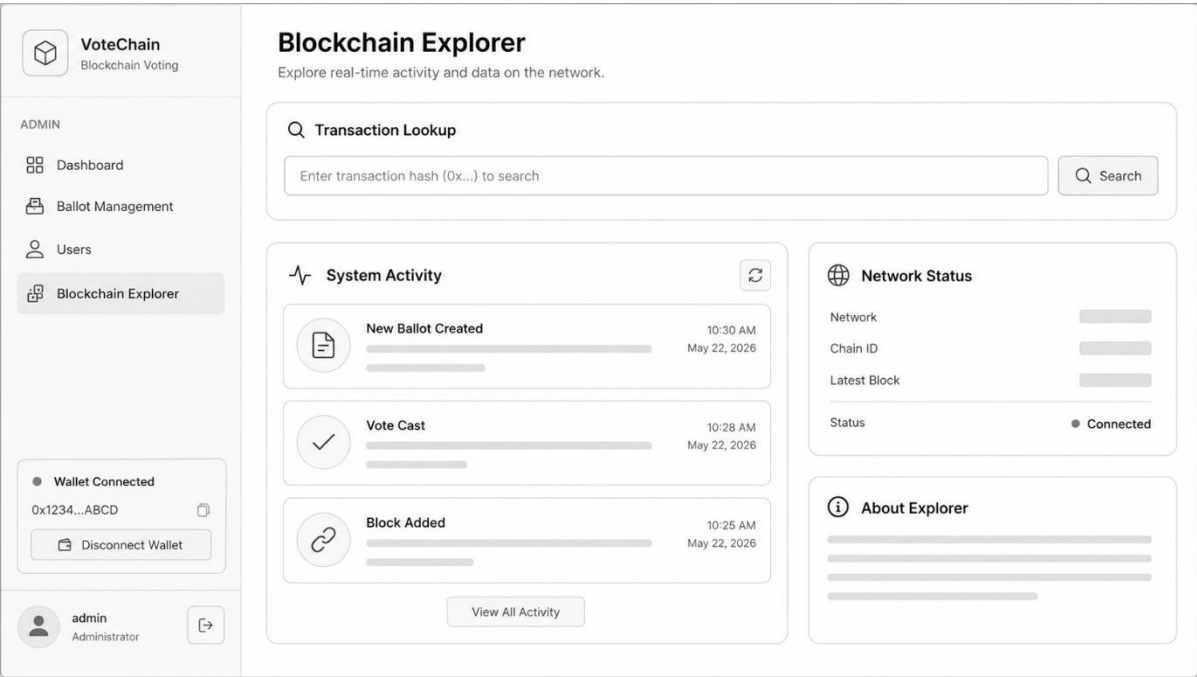


Figure 19: Blockchain Explorer Interface Design

3.9.6 Security and Privacy Design

No	Security / Privacy Area	Mechanism	Implementation in VoteChain	Purpose
1	Hybrid Authentication and Authorization	Web2 + Web3 authentication model	The system combines account login with MetaMask wallet connection. Users first authenticate through email/password, then connect a Web3 wallet to perform blockchain-related actions.	Ensures that users are verified at both the application layer and blockchain transaction layer.
1.1	Web2 Authentication	Password hashing and JWT session management	User passwords are hashed using bcrypt before being stored in SQLite. After login, the system issues a JSON Web Token (JWT), which is	Prevents plaintext password exposure and blocks unauthorized access

			included in HTTP headers to protect API requests.	to protected API endpoints.
1.2	Web3 Wallet Authentication	MetaMask transaction signing	Users must connect MetaMask before voting or performing administrative blockchain actions. Transactions are signed directly with the private key stored inside the user's wallet.	Ensures transaction ownership without exposing private keys to the application or backend server.
2	Private Key Protection	Client-side wallet custody	VoteChain does not store, request, or transmit users' private keys. Private keys remain inside MetaMask.	Eliminates the risk of private key leakage from the server side.
3	Smart Contract Access Control	onlyOwner modifier	Sensitive administrative functions such as createElection(), addCandidate(), and finalizeElection() can only be executed by the smart contract owner wallet.	Prevents unauthorized users from creating, modifying, or finalizing elections.
4	Temporal Integrity	electionActive modifier	The smart contract only allows voting when the current time is between startTime and endTime, and the election has not been finalized.	Ensures that votes are accepted only during the valid voting period.
5	Voter Anonymity	Anonymous vote receipt generation	When a voter casts a vote, the system generates a voteHash using Keccak256 based on election ID,	Allows voters to verify participation while preventing

			candidate ID, voter address, and timestamp.	direct exposure of their voting choice.
6	Choice Privacy	No public voter-to-candidate mapping	The smart contract increases the selected candidate's voteCount directly but does not publicly store a mapping between voter address and selected candidate ID.	Prevents other users, Admins, or observers from directly identifying which candidate a voter selected.
7	Double-Voting Prevention	notVoted modifier and hasVoted mapping	Before accepting a vote, the smart contract checks hasVoted[electionId][msg.sender]. If the wallet has already voted, the transaction is rejected.	Ensures that each wallet can vote only once in each election.
8	Transaction Rejection for Invalid Voting	Smart contract require() validation	If a voter tries to vote twice, the smart contract executes require(!hasVoted[_electionId][msg.sender], "You have already voted"); and reverts the transaction.	Blocks fraudulent voting attempts at the blockchain level, independent of frontend or backend logic.
9	Data Integrity	Blockchain immutability	Once election creation, voting, or finalization transactions are confirmed on-chain, they cannot be modified or deleted.	Protects election records from tampering, deletion, or unauthorized modification.
10	Independent Auditability	Blockchain events and transaction history	The smart contract emits events such as VoteCast for successful voting actions. Third parties can scan blockchain events and	Allows independent auditing without relying solely on the application's SQLite database.

			independently verify election activity.	
11	Result Integrity	On-chain vote counting	Vote counts are updated directly in the smart contract through the candidate's voteCount field.	Ensures that final results are based on blockchain records rather than editable off-chain data.
12	Off-chain Data Protection	Sensitive data separation	Personal user information and supporting metadata are stored off-chain in SQLite, while critical voting transactions are stored or referenced on-chain.	Avoids exposing sensitive personal data directly on a public blockchain while maintaining verifiability.

Table 20: Security and Privacy Design

4. Implementation and Software Demo

4.1 Development Environment

Category	Component / Library	Version	Role and Main Function
Operating System & Basic Environment	Operating System (OS)	Windows 10/11 / macOS / Linux	System environment used for development and testing.
	Node.js Runtime	v18.x or v20.x (LTS)	JavaScript runtime environment used to run the Backend and Hardhat.
	npm Package Manager	v9.x or v10.x	Package manager used to install project dependencies.
Database	SQLite3	^5.1.7	Embedded file-based database management system used to store local data.

Blockchain (Web3)	Hardhat Framework	^2.28.6	Tool used to compile, test, and deploy Smart Contracts.
	Hardhat Toolbox	^5.0.0	Hardhat plugin package that supports advanced testing and blockchain network management.
	Solidity Compiler	^0.8.24	Compiler used to convert Solidity source code into EVM-compatible bytecode.
	Sepolia Testnet	Ethereum Test Network	Ethereum testnet used to deploy and test the Smart Contract in a realistic environment.
	MetaMask Extension	Browser Extension	Web3 browser wallet used by voters and Admins to digitally sign blockchain transactions.
Backend Stack	Express.js	^4.19.2	Web framework used to build RESTful APIs and coordinate business logic.
	Ethers.js (Backend)	^6.11.1	Library used to connect to and call Smart Contract functions on the Sepolia network.
	JSON Web Token (JWT)	^9.0.2	Used to generate and verify JWT tokens for API access control.
	BcryptJS	^2.4.3	Used to hash voter passwords before storing them in the SQLite database.

	Nodemon (Dev Tool)	^3.1.0	Automatically restarts the API server when source code changes are detected.
	CORS	^2.8.5	Enables resource sharing between the Frontend and Backend origins.
Frontend Stack	React	^18.3.1	Library used to build the Single Page Application (SPA) user interface.
	Vite (Build Tool)	^5.2.11	Fast development server and optimized code bundling tool.
	Ethers.js (Frontend)	^6.11.1	Used to interact directly with the user's MetaMask wallet during voting operations.
	React Router DOM	^6.23.0	Handles routing and navigation between application pages.
	Axios	^1.6.8	Sends HTTP requests such as GET, POST, and PUT from the client to the API server.
	Recharts	^2.12.7	Used to draw visual statistical charts for candidate vote counts.
	Lucide React	^0.378.0	Modern minimalist vector icon library used in the user interface.
	React Hot Toast	^2.4.1	Displays smooth popup toast notifications to improve user experience.

Tools & Utilities	VS Code IDE	Microsoft	Recommended source code editor for the project.
	Solidity VS Code Extension	Nomic Foundation	Supports Solidity syntax highlighting and code formatting in the editor.
	SQLite Viewer Extension	Florian Albrecht / qwtel	Extension used to view SQLite database tables directly inside VS Code.

Table 21: Development Environment

4.2. Smart Contract Implementation

This section presents the main functions of the Voting smart contract.

Function	Description
createElection()	Creates and sets up a new election. This function can only be executed by the Admin.
addCandidate()	Adds a new candidate to a specific election. This function can only be executed by the Admin.
finalizeElection()	Officially closes and finalizes an election in order to lock the results. This function can only be executed by the Admin.
castVote()	Allows a voter to vote for the selected candidate. The function automatically generates a hashed anonymous vote receipt to protect voter privacy.
getResults()	Retrieves the list of candidates and the actual total number of votes received by each candidate.
getVoteReceipt()	Retrieves the voter's anonymous hashed voting receipt for transparent auditing and verification.
hasVoterVoted()	Checks whether a voter, identified by wallet address, has already voted in a specific election.

getCandidate()	Retrieves detailed information of a specific candidate, including ID, name, description, and vote count.
getElection()	Retrieves detailed election configuration information, including name, voting time, number of candidates, and number of voters.
getElectionStatus()	Checks the current status of an election, such as pending, active, ended, or finalized.

Table 22: Smart Contract Functions

4.3 Main Screenshots

- Login/Register Screen:

This screen allows users to log in or register an account before accessing the VoteChain system. Users enter their email or username and password, then submit the form to authenticate and continue to the appropriate dashboard based on their role.

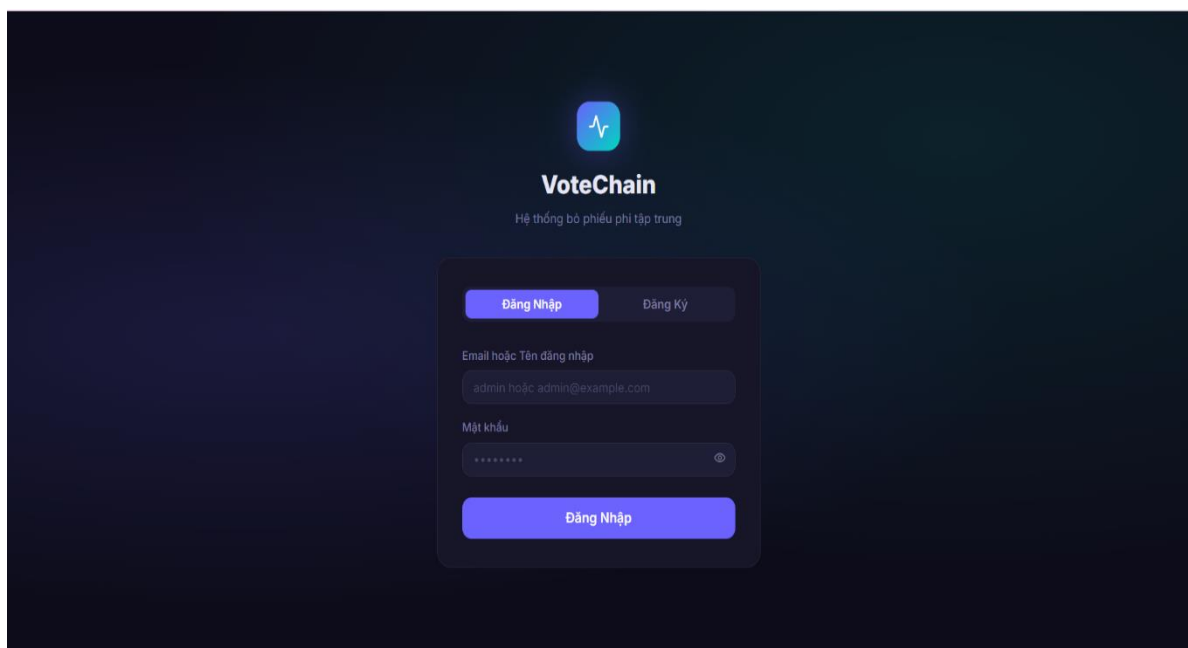


Figure 20: Login/Register Screen

- Admin Dashboard Screen:

This screen provides an overview of the VoteChain system for the Admin. It displays key statistics such as total elections, registered voters, total votes, and users. It also includes quick actions for creating elections, managing voters, and opening the Blockchain Explorer, along with charts and recent election records for monitoring system activity.

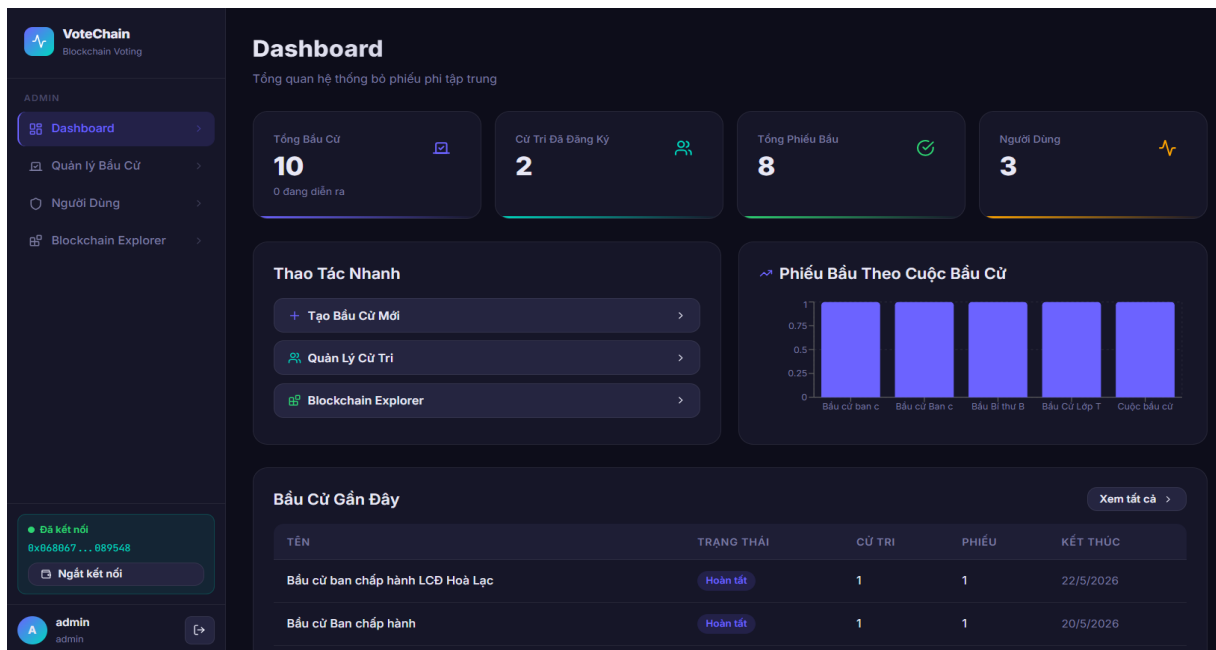


Figure 21: Admin Dashboard Screen

- Election Management Screen:

This screen allows the Admin to manage all elections in the VoteChain system. The Admin can view election details, filter elections by status, create a new election, check voting time, number of candidates, voters, and votes, and open the candidate management function for each election.

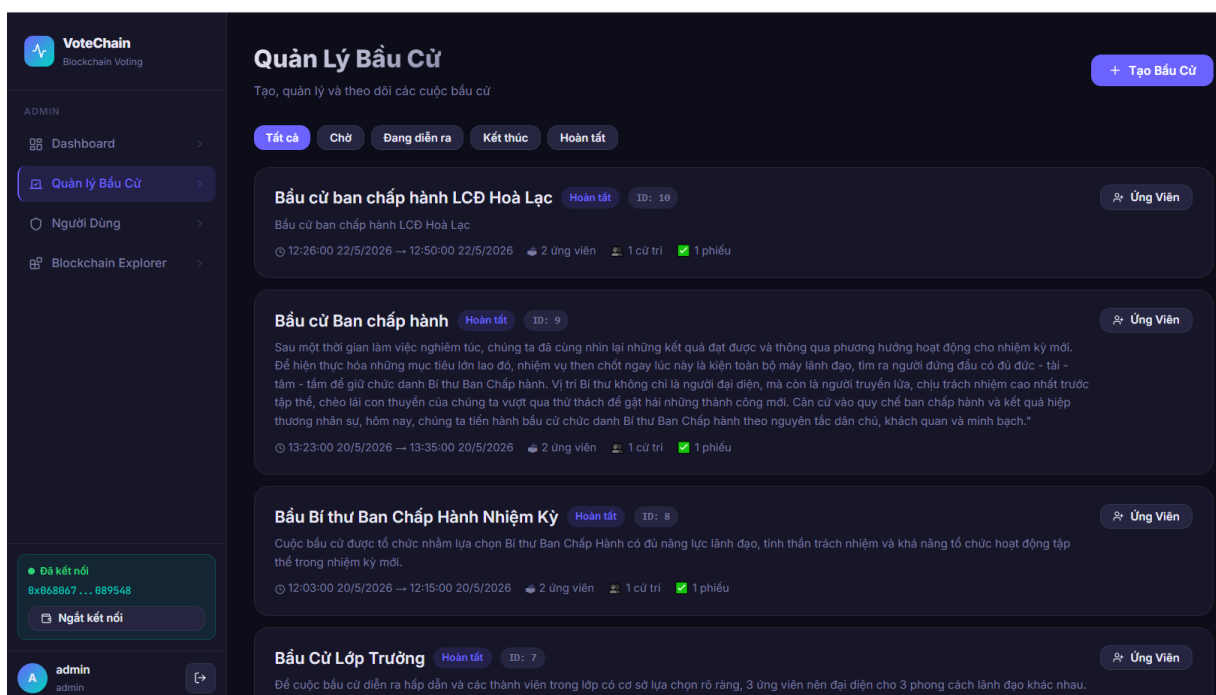


Figure 22: Election Management Screen

- User Management Screen:

This screen allows the Admin to view and manage all user accounts in the VoteChain system. It displays key user information such as username, email, role, linked MetaMask wallet address, and account creation date. This helps the Admin monitor registered voters, identify Admin/Voter roles, and check whether each user has connected a wallet.

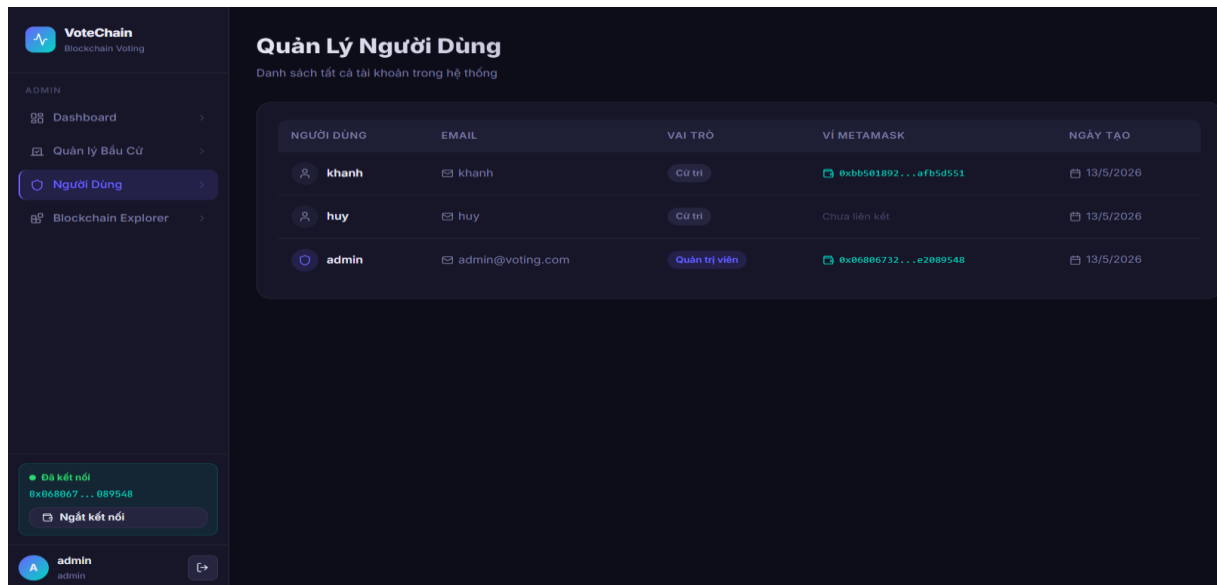


Figure 23: User Management Screen

- Create Election Modal:

This screen allows the Admin to create a new election by entering the election name, description, start date/time, and end date/time. The background is blurred to keep user focus on the form, and the Admin can either cancel the action or submit the election creation request.

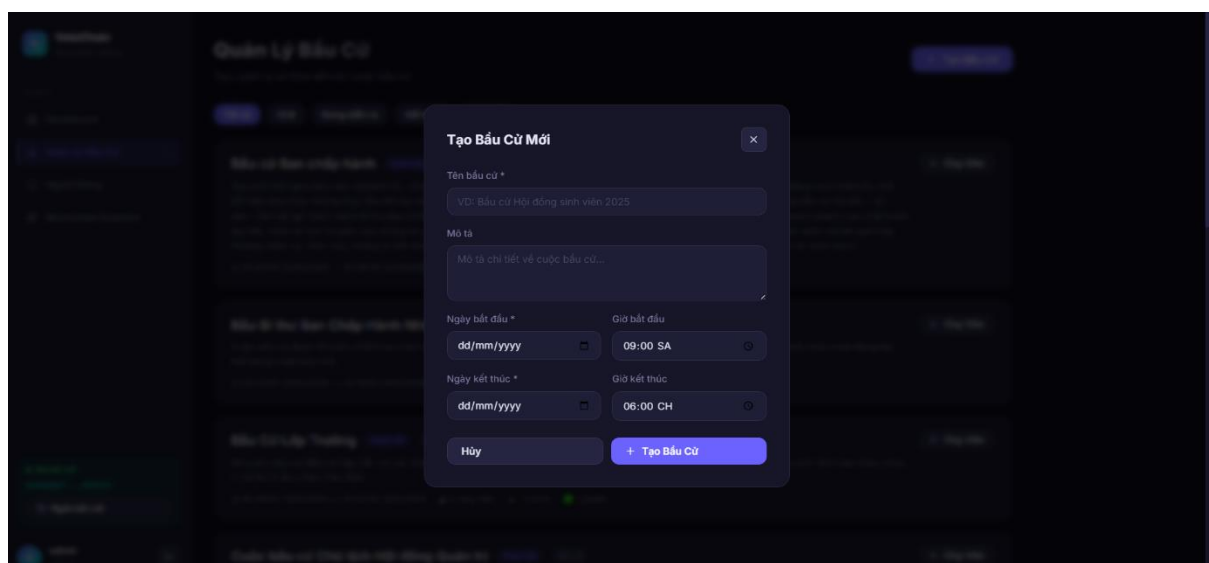


Figure 24: Create Election Modal

- Add Candidates / Election Detail Screen:

This screen allows the Admin to view detailed election information, including voting time, total votes, and voter participation. The Admin can also add new candidates by entering candidate information and submitting it to the system. This page supports election preparation before voting begins and helps the Admin monitor election status.

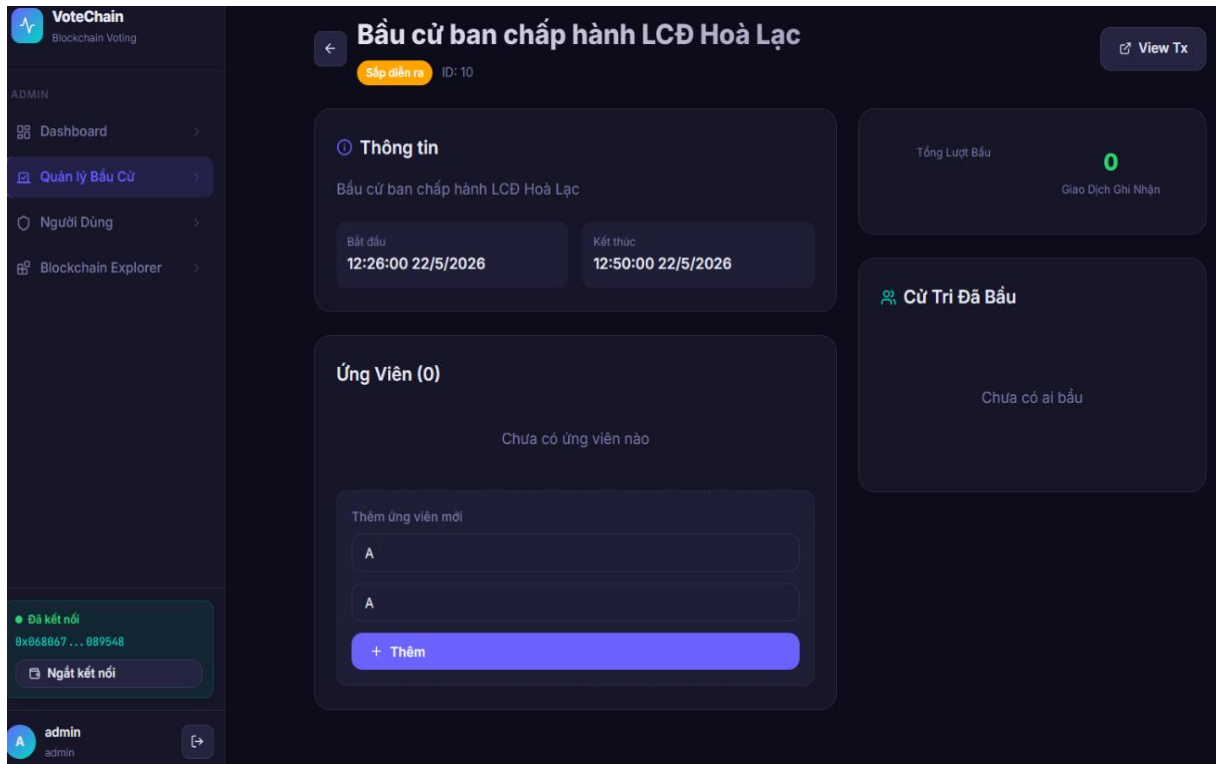


Figure 25: Add Candidates / Election Detail Screen

- Voting Screen:

This screen allows the Voter to participate in an active election by reviewing the election title and candidate list, selecting one candidate, and submitting the vote. The system requires a connected wallet before voting, ensuring that the vote can be signed and recorded securely on the blockchain.

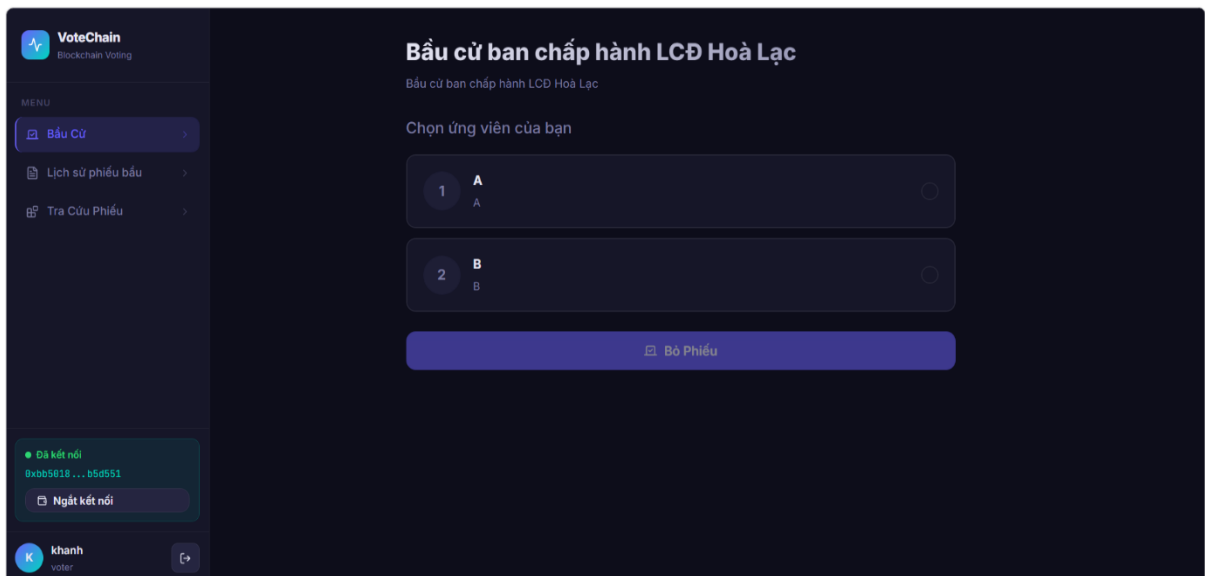


Figure 26: Voting Screen

- Vote History Screen:

This screen allows the Voter to view all recorded voting transactions on the Sepolia Testnet. Each record includes the election name, transaction hash, anonymous vote hash, block number, and voting time. Users can search by transaction hash, vote hash, or election ID, copy transaction data, and open the transaction for independent verification.

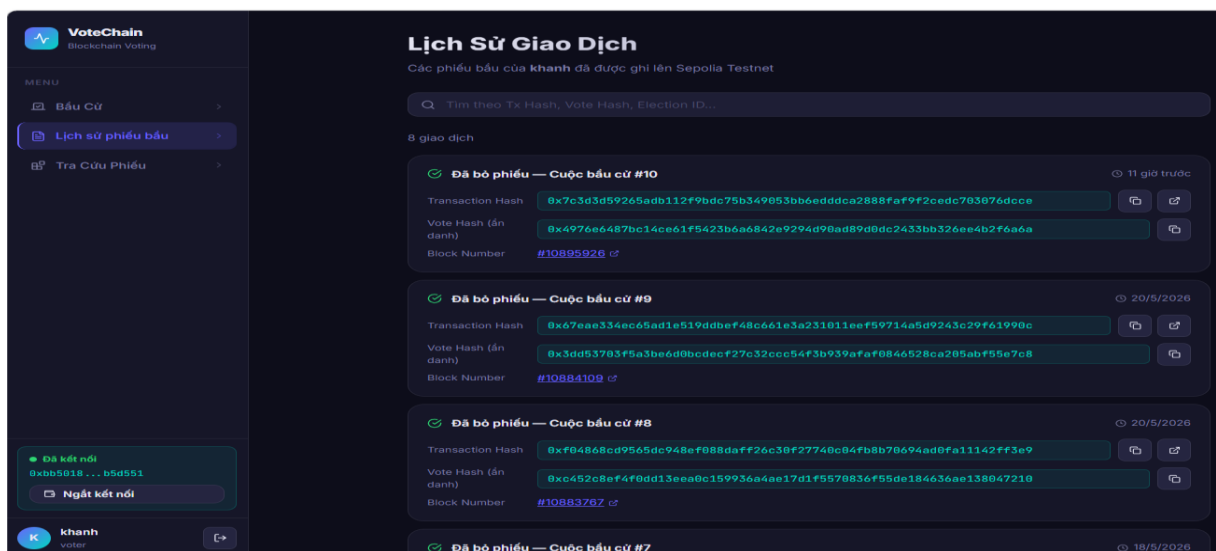


Figure 27: Vote History Screen

- Election Results Screen:

This screen displays the final result of an election after voting is completed. It shows the winning candidate, total votes, vote percentage, bar chart, pie chart, and ranking table. Users can refresh the data to view the latest results and verify the election outcome clearly.



Figure 28: Election Results Screen

- Blockchain Explorer Screen:

This screen allows users to search and verify blockchain transactions on the Sepolia Testnet. It displays recent system activities, transaction hashes, network status, chain ID, and current block number. Users can copy transaction hashes or open them externally to confirm election-related actions such as voting, election creation, and result verification.

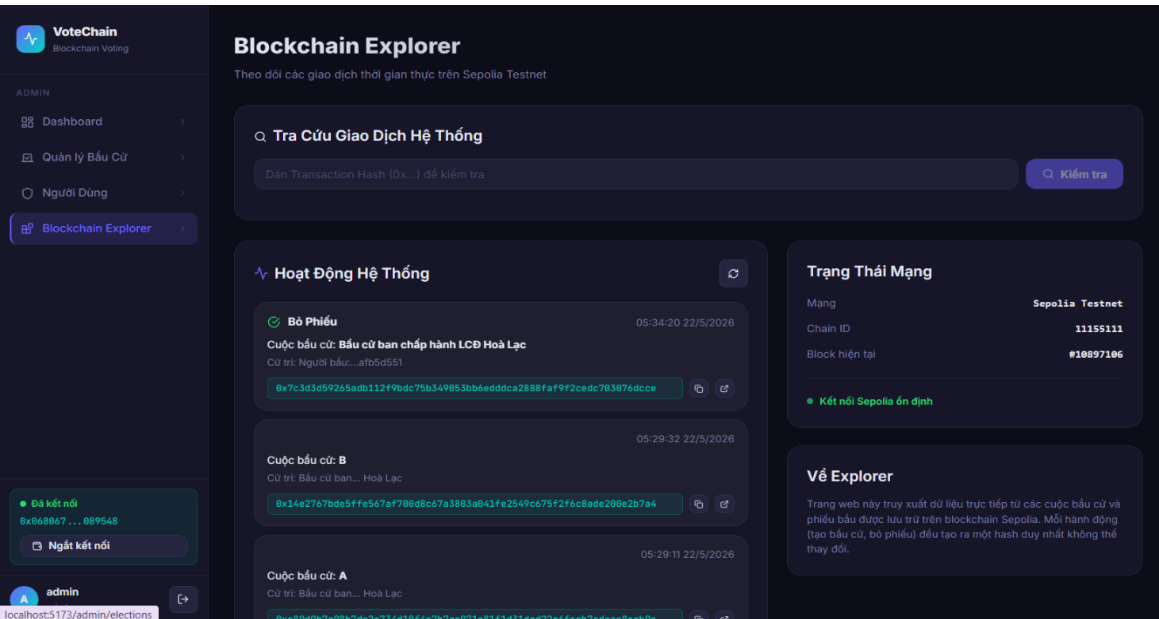


Figure 29: Blockchain Explorer Screen

4.4 Demo Scenario

The objective of this demo scenario is to execute the complete lifecycle of a hypothetical election named “Election of the Hoà Lạc Inter-branch Youth Union Executive Committee”, with two candidates: Nguyễn Văn A and Trần Thị B. The process is divided into the following 9 specific steps:

Step	Actor	User Action	System Processing	Expected UI Output
1	Admin	Accesses /login, enters the Admin account username admin and password, then clicks Login.	The Backend API verifies the account in the users table in SQLite using bcrypt. If the credentials are valid, the system issues a JWT token and stores it on the client for authorization.	Login is successful, and the system automatically redirects the Admin to the Dashboard page.
2	Admin	Clicks the Connect Wallet button at the bottom-left corner of the Sidebar.	The React frontend uses ethers.js to send a wallet connection request through the window.ethereum object provided by MetaMask.	MetaMask opens a popup requesting wallet connection. After approval, the Sidebar displays the shortened Admin wallet address, such as 0x868067...089548, with the status Connected.
3	Admin	Goes to Election Management (/admin/elections), clicks Create New Election, enters the election name,	The frontend sends a transaction to call the createElection() function in the Voting.sol smart contract. After the	MetaMask displays a gas fee estimation popup and asks the Admin to sign the transaction. After success, the new

		description, start/end time, and confirms creation.	transaction is mined on the Sepolia Testnet, the Backend synchronizes the election data and transaction hash (tx_hash) into the elections table in SQLite.	election appears in the election list with the Active status.
4	Admin	Opens the details of the newly created election, selects Add Candidate, and enters candidate information: Candidate 1: Nguyễn Văn A; Candidate 2: Trần Thị B.	The frontend executes the addCandidate() function on the smart contract using the Admin's digital signature. After the Sepolia transaction is completed, candidate information is synchronized into the candidates table in SQLite.	The transaction is confirmed on-chain, and the interface updates the candidate list of the election.
5	Admin	Goes to Users (/admin/users), selects Authorize Voter, enters the public MetaMask wallet address of the test voter, for example 0xAfB5D551..., and clicks Save.	The system stores this wallet address in the valid voter list in the authorized_voters table in SQLite for access control.	The system displays the message Voter added successfully, and the authorized voter list shows the newly added wallet address.
6.1	Voter	Logs in with a voter account and clicks Connect Wallet on the	The Backend API authenticates the voter account and issues a	MetaMask displays a wallet connection popup. After successful

		voter interface. The connected MetaMask wallet must be the one authorized by the Admin in Step 5.	JWT session token. The frontend then sends a connection request to the voter's MetaMask wallet.	connection, the screen shows the connected status and the voter's wallet address.
6.2	Voter	Opens the list of active elections and selects Bầu cử ban chấp hành LCD Hoà Lạc.	The Backend API queries SQLite and returns the election information and candidate list associated with that election to the client.	The interface displays the election details and the two candidates, Nguyễn Văn A and Trần Thị B, together with the Vote button.
6.3	Voter	Selects Nguyễn Văn A and clicks Vote.	The frontend calls the castVote() function in the smart contract. After the transaction is completed, the contract increases the selected candidate's vote count by 1, sets hasVoted = true for the voter's wallet, generates an anonymous voteHash, and emits the VoteCast event.	MetaMask requests transaction confirmation and charges gas. After success, the system displays a digital receipt popup with the anonymous vote hash (voteHash).
7	Admin / Voter	Opens the Blockchain Explorer menu in the Sidebar to check election transparency.	The Blockchain Explorer page listens to and synchronizes events from the smart contract on the Sepolia network.	The recent transaction appears in the activity timeline, showing the Vote Cast activity, the shortened voter wallet address, and a direct link to Etherscan.

8	Admin	After the voting period ends, the Admin opens the election management page and clicks Finalize Election.	The system sends a transaction to call the finalizeElection() function in the smart contract. After the Sepolia transaction succeeds, the election status changes to finalized, and the voting function is locked.	MetaMask asks the Admin to confirm the finalization transaction. The election status changes to Finalized, and the voter's voting button is disabled.
9	Admin / Voter	Opens the detail page of the finalized election to view the final result.	The system calls the getResults() function from the smart contract to directly retrieve the on-chain vote count of each candidate and displays it through chart components.	The system displays accurate and transparent final results using visual percentage charts through Recharts, allowing all users to view the outcome.

Table 23: Demo Scenario

4.5 Testing

Test Case ID	Feature	Actor	Test Scenario	Key Steps	Expected Result	Status
TC01	User Authentication	Admin / Voter	Verify that users can log in securely and access the correct dashboard.	1. Go to /login. 2. Enter valid username and password.	The backend verifies credentials, generates a JWT token, stores it on the client, and redirects the user based on role. Unauthorized	Pass

				3. Click Login.	access to Admin pages is blocked.	
TC02	Web3 Wallet Integration	Admin / Voter	Verify that users can connect MetaMask and retrieve their wallet address.	1. Click Connect Wallet. 2. Select wallet account in MetaMask. 3. Confirm connection.	MetaMask popup opens successfully. After approval, the system displays the connected status and shortened public wallet address.	Pass
TC03	Create Election On-chain	Admin	Verify that Admin can create a new election on Sepolia and sync data to the local database.	1. Open Election Management. 2. Click Create New Election. 3. Enter name, description, start time, and end time. 4. Confirm transaction in MetaMask.	The smart contract executes createElection(). After the transaction is mined, the election metadata and tx_hash are saved into SQLite.	Pass
TC04	Add Candidate	Admin	Verify that Admin can add candidates	1. Open election detail page.	The smart contract executes addCandidate(). Candidate data is	Pass

			before voting starts.	2. Click Add Candidate. 3. Enter candidate name and description. 4. Sign transaction in MetaMask.	stored on-chain, synced to SQLite, and displayed in the candidate list.	
TC05	Authorize Voter	Admin	Verify that Admin can authorize voter wallet addresses.	1. Go to User Management. 2. Enter voter public wallet address. 3. Click Save.	The wallet address is saved into the authorized_voters table, and a success notification is displayed.	Pass
TC06	Cast Anonymous Vote	Voter	Verify that an authorized voter can cast a vote and receive an anonymous receipt.	1. Log in as Voter. 2. Connect authorized MetaMask wallet. 3. Open active election. 4. Select candidate	The smart contract executes castVote(), increases the candidate's voteCount, marks the wallet as voted, generates voteHash, and displays the digital receipt.	Pass

				and click Vote. 5. Confirm gas fee in MetaMask.		
TC07	Double- Voting Prevention	Voter	Verify that the same wallet cannot vote twice in one election.	1. Use a wallet that has already voted. 2. Try to vote again in the same election.	The frontend disables or blocks the vote action. If called directly, the smart contract reverts the transaction with “You have already voted.”	Pass
TC08	Blockchain Explorer Verification	Admin / Voter	Verify that election and voting transactions can be audited.	1. Open Blockchain Explorer. 2. Review live transaction feed or search by transaction hash. 3. Open the external Etherscan link.	The transaction appears with correct timestamp, election name, wallet address, transaction hash, and direct Sepolia Etherscan link.	Pass
TC09	Finalize Election &	Admin	Verify that Admin can finalize an	1. Open Admin election	The smart contract executes finalizeElection(),	Pass

	Render Statistics		election and display final results.	management page. 2. Click Finalize Election. 3. Sign transaction in MetaMask. 4. Open result page.	election status becomes finalized, further voting is disabled, and results are displayed using charts.	
--	-------------------	--	-------------------------------------	---	--	--

Table 24: Testing Results

5. Conclusion

5.1. Advantages of the Proposed System

VoteChain provides a modern and reliable approach to electronic voting by combining a web-based application with blockchain-based verification. Unlike traditional voting systems that depend heavily on a central authority, VoteChain records critical election activities on-chain, including election creation, vote submission, and election finalization. Once these actions are confirmed on the blockchain, they cannot be modified or deleted. This immutability strengthens the integrity of the election process and allows election data to be independently audited by users, administrators, or external observers.

One of the most important advantages of the system is transparency. Each major transaction is associated with a transaction hash, which can be used to verify the action through a blockchain explorer. This means that users do not need to rely entirely on the internal database or the election organizer to trust the result. Instead, they can independently check whether an election action has actually been recorded on the blockchain. This improves trust and reduces the possibility of hidden manipulation.

Another key advantage is voter privacy. VoteChain generates anonymous vote receipts that allow voters to confirm their participation without directly revealing which candidate they selected. This design helps balance two important requirements in electronic voting:

transparency and ballot confidentiality. Voters can verify that their vote has been recorded, while their personal voting choice remains protected.

The system also provides strong protection against double voting. Each wallet address can vote only once in a specific election, and this rule is enforced at the smart contract level. Because the rule is implemented on-chain, it cannot be bypassed through the frontend or backend. This improves election fairness and ensures that the final vote count accurately reflects valid participation.

In addition, VoteChain improves usability through a clear and role-based interface. Admin users can create elections, add candidates, finalize elections, monitor statistics, and review blockchain transactions through the dashboard. Voter users can view active elections, select candidates, cast votes, receive receipts, verify transactions, and review their voting history. Result charts and blockchain explorer integration also make the system easier to understand and more transparent for both technical and non-technical users.

Overall, the proposed system demonstrates how blockchain technology can be applied to improve electronic voting. It provides a voting platform that is transparent, tamper-resistant, auditable, privacy-preserving, and easier to manage than many traditional voting processes.

5.2. Limitations

Although VoteChain successfully demonstrates the feasibility of a decentralized voting platform, the current version still has several limitations. First, the system is deployed on the Sepolia Testnet rather than a production blockchain network. This is suitable for development, testing, and academic demonstration because it avoids real transaction costs. However, performance, gas fees, transaction confirmation speed, and network stability may be different when the system is deployed in a real production environment.

Second, the system depends on MetaMask for wallet connection and transaction signing. While MetaMask is widely used in Web3 applications, it may still be unfamiliar to non-technical users. Voters must know how to install the extension, connect a wallet, switch to the correct network, and confirm blockchain transactions. This requirement may create a usability barrier for users who have no prior experience with blockchain wallets.

Third, the current database layer uses SQLite, which is simple, lightweight, and suitable for testing or small-scale deployment. However, SQLite may not be ideal for large-scale elections with many users, many concurrent requests, and high transaction volume. For

production use, the system would need a more scalable database solution such as PostgreSQL or MySQL, along with stronger backup, recovery, and concurrency mechanisms.

Fourth, the current system mainly supports basic election scenarios. It allows admins to create elections, add candidates, authorize voters, cast votes, finalize elections, and view results. However, real-world elections may require more complex features such as multi-round voting, voter identity verification, role-based election committees, organization-level election settings, audit report generation, and legal compliance mechanisms.

Another limitation is that voter anonymity is implemented through anonymous vote hashes, but the system still uses public wallet addresses for participation tracking. Although the selected candidate is not directly exposed, advanced privacy-preserving techniques may be needed for highly sensitive elections. Technologies such as zero-knowledge proofs or blind signatures could provide stronger privacy in future versions.

Finally, the current system has not yet been deployed as a full production service with a real domain, cloud hosting, monitoring system, automated backup, or disaster recovery strategy. Therefore, VoteChain should currently be considered a functional prototype or proof-of-concept rather than a complete production-ready election platform.

5.3. Future Work

In the future, VoteChain can be improved in several important directions to become more scalable, secure, and practical for real-world use. First, the system can be deployed to a production-ready blockchain environment or a Layer-2 network. Layer-2 solutions can reduce transaction fees and improve transaction speed, making the platform more suitable for larger elections with many voters.

Second, the authentication and voter verification mechanism can be enhanced. Future versions may include email verification, student ID verification, organization-based approval, national digital identity integration, or Know Your Customer-style identity checks depending on the election context. These improvements would help ensure that only eligible voters are allowed to participate.

Third, the database layer can be upgraded from SQLite to a more scalable database system such as PostgreSQL. This would allow the system to support more users, more elections, larger datasets, and higher concurrency. In addition, caching, indexing, backup scheduling, and monitoring tools can be added to improve long-term reliability.

Fourth, the user experience should be further simplified, especially for voters who are not familiar with blockchain. The system can provide clearer wallet connection instructions, step-by-step voting guidance, better error messages, and more intuitive transaction status indicators. A mobile-responsive interface or native mobile application can also be developed to make voting more accessible.

Fifth, the privacy mechanism can be strengthened by applying advanced cryptographic methods. For example, zero-knowledge proofs could allow voters to prove eligibility and vote validity without revealing sensitive information. This would make the system more suitable for high-stakes elections where privacy requirements are stricter.

Sixth, the system can add more administrative and auditing features. Future versions may include downloadable audit reports, QR-code-based vote receipt verification, detailed election activity logs, role-based election committees, and automated result certification. These features would make VoteChain more useful for organizations, schools, communities, and institutions.

Finally, additional convenience features such as multi-language support, email notifications, SMS reminders, candidate profile pages, accessibility improvements, and post-voting NFT badges can be considered. These improvements would make VoteChain more complete, user-friendly, and adaptable to different voting environments.

In conclusion, VoteChain has successfully demonstrated the potential of blockchain technology in building a transparent, secure, and verifiable electronic voting system. While the current version still has limitations, it provides a strong foundation for future development and real-world adoption.

6. References

- Jafar, U., Aziz, M. J. A., & Shukur, Z. (2021). Blockchain for electronic voting system- Review and open research challenges. *Sensors*, 21(17), 5874.
- Jafar, U., Ab Aziz, M. J., Shukur, Z., & Hussain, H. A. (2022). A systematic literature review and meta-analysis on scalable blockchain-based electronic voting systems. *Sensors*, 22(19), 7585.
- McCorry, P., Shahandashti, S. F., & Hao, F. (2017). A smart contract for boardroom voting with maximum voter privacy. In *Financial Cryptography and Data Security* (pp. 357–375). Springer.
- Hardwick, F. S., Gioulis, A., Akram, R. N., & Markantonakis, K. (2018). E-voting with blockchain: An e-voting protocol with decentralisation and voter privacy. In *2018 IEEE International Conference on Internet of Things, Green Computing and Communications, Cyber, Physical and Social Computing, Smart Data* (pp. 1561–1567). IEEE.
- Shahzad, B., & Crowcroft, J. (2019). Trustworthy electronic voting using adjusted blockchain technology. *IEEE Access*, 7, 24477–24488.
- Gao, S., Zheng, D., Guo, R., Jing, C., & Hu, C. (2019). An anti-quantum e-voting protocol in blockchain with audit function. *IEEE Access*, 7, 115304–115316.
- Khan, K. M., Arshad, J., & Khan, M. M. (2020). Investigating performance constraints for blockchain-based secure e-voting system. *Future Generation Computer Systems*, 105, 13–26.
- Khan, K. M., Arshad, J., & Khan, M. M. (2018). Secure digital voting system based on blockchain technology. *International Journal of Electronic Government Research*, 14(1), 53–62.
- Alvi, S. T., Uddin, M. N., Islam, L., & Ahamed, S. (2022). DVTChain: A blockchain-based decentralized mechanism to ensure the security of digital voting system. *Journal of King Saud University - Computer and Information Sciences*, 34(9), 6855–6871.
- Bartolucci, S., Bernat, P., & Joseph, D. (2018). SHARVOT: Secret SHARe-based VOTing on the blockchain. In *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain* (pp. 30–34). ACM.

Yu, B., Liu, J. K., Sakzad, A., Nepal, S., Steinfeld, R., Rimba, P., & Au, M. H. (2018). Platform-independent secure blockchain-based voting system. In *Information Security: 21st International Conference, ISC 2018* (pp. 369–386). Springer.

Hjalmarsson, F. P., Hreidarsson, G. K., Hamdaqa, M., & Hjalmtysson, G. (2018). Blockchain-based e-voting system. In *2018 IEEE 11th International Conference on Cloud Computing* (pp. 983–986). IEEE.

Lai, W. J., Hsieh, Y. C., Hsueh, C. W., & Wu, J. L. (2018). DATE: A decentralized, anonymous, and transparent e-voting system. In *2018 1st IEEE International Conference on Hot Information-Centric Networking* (pp. 24–29). IEEE.

Yi, H. (2019). Securing e-voting based on blockchain in P2P network. *EURASIP Journal on Wireless Communications and Networking*, 2019, Article 137.

Yavuz, E., Koç, A. K., Çabuk, U. C., & Dalkılıç, G. (2018). Towards secure e-voting using Ethereum blockchain. In *2018 6th International Symposium on Digital Forensic and Security* (pp. 1–7). IEEE.

Hanifatunnisa, R., & Rahardjo, B. (2017). Blockchain-based e-voting recording system design. In *2017 11th International Conference on Telecommunication Systems Services and Applications* (pp. 1–6). IEEE.

Kshetri, N., & Voas, J. (2018). Blockchain-enabled e-voting. *IEEE Software*, 35(4), 95–99.

Panja, S., & Roy, B. K. (2021). A secure end-to-end verifiable e-voting system using blockchain. *Journal of Information Security and Applications*, 60, 102891.

Sharp, M., Akram, R. N., & Markantonakis, K. (2024). Blockchain-based e-voting mechanisms: A survey and open challenges. *Blockchains*, 2(4), 441–474.

Daraghmi, E., Hamoudi, A., & Abu Helou, M. (2024). Decentralizing democracy: Secure and transparent e-voting systems with blockchain technology in the context of Palestine. *Future Internet*, 16(11), 388.